

Platform Engineering Handbook

From Fundamentals to Production-Grade Azure Lab

Gopal Khandale

2026

Contents

Preface	11
What this handbook is	11
How this handbook is structured	11
How each chapter is structured	11
Concept	11
Why it matters	11
How it works	11
Gotchas	12
In our lab	12
Interview talking points	12
Exercises	12
How to use this handbook	12
Part I — Foundations	13
Chapter 1: The Cloud and Why We’re Here	14
1.1 What the cloud actually is	14
Concept	14
Why it matters	14
How it works	14
Service models	15
Picking the right model	15
In our lab	16
Gotchas	16
Interview talking points	16
Exercises	16
1.2 The three big clouds	17
Concept	17
The contenders	17
Picking a cloud	17
Why this book uses Azure	18
Interview talking points	18
1.3 Regions and availability zones	18
Concept	18
Why this matters	18
How regions and AZs are organized	18

Common Azure regions	19
Resource groups	19
In our lab	19
Cost model	20
Interview talking points	20
Exercises	20
Chapter 2: Linux for Platform Engineers	22
2.1 Why Linux	22
Concept	22
What you need to know	22
2.2 The shell	22
Concept	22
Key shell features	23
Useful shell tips	23
2.3 Essential commands	23
File navigation	23
Reading files	24
Searching files	24
Text processing	24
Processes	24
Network	25
File transfer	25
2.4 Permissions	25
Concept	25
Numeric representation	26
Common commands	26
SSH key permissions	26
2.5 SSH and remote work	26
Concept	26
Generate a key pair	26
SSH config file	27
LocalForward (port tunneling)	27
How LocalForward works	27
Useful SSH tips	28
2.6 Environment variables	28
Concept	28
Setting variables	28
Persisting across logins	28
2.7 systemd	29
Concept	29
Key commands	29
When you'll use systemd	29
Service status meanings	29
2.8 Exercises	30
Chapter 3: Git and Version Control	31
3.1 What Git is	31

Concept	31
Why Git matters	31
3.2 The three states of a file	31
Concept	31
Visualizing state transitions	32
3.3 Daily commands	32
Setup	32
Starting a repo	32
Checking status and changes	32
Making changes	32
Working with branches	33
Synchronizing with remote	33
3.4 Branching strategies	33
Concept	33
Git Flow (heavy, old-fashioned)	33
GitHub Flow (modern, recommended)	33
Trunk-Based Development	34
3.5 Pull Requests	34
Concept	34
Why PRs matter	34
Anatomy of a good PR	34
3.6 Merge conflicts	35
Concept	35
What a conflict looks like	35
Resolving conflicts	35
Tips for fewer conflicts	36
3.7 Rewriting history (carefully)	36
Concept	36
The commands	36
The cardinal rule	36
When rewriting is safe	36
Common pattern: clean up before push	36
3.8 .gitignore	37
Concept	37
Common entries	37
Critical for security	38
3.9 Git in CI/CD	38
Concept	38
Why we use short SHA as image tags	38
A useful workflow	38
3.10 Interview talking points	39
3.11 Exercises	39
Chapter 4: Networking Essentials	41
4.1 Why networking matters	41
Concept	41
4.2 The OSI model (simplified)	41
Concept	41

4.3 IP addresses and CIDR	42
Concept	42
How CIDR sizing works	42
Our lab’s networking	42
4.4 Public vs Private IPs	42
Concept	42
How private IPs reach the internet	42
4.5 DNS	43
Concept	43
Common record types	43
DNS in Kubernetes	43
4.6 TCP and UDP, ports	44
Concept	44
TCP vs UDP	44
Common ports to memorize	44
4.7 Firewalls and NSGs	44
Concept	44
Azure’s default rules	45
The explicit deny gotcha	45
How NSG rules are evaluated	46
4.8 Load Balancers	46
Concept	46
Visual	46
In our lab	47
4.9 TLS and HTTPS	47
Concept	47
The TLS handshake	47
Certificates	47
mTLS (mutual TLS)	48
4.10 Interview talking points	48
4.11 Exercises	48
Chapter 5: Containers and Docker	49
5.1 The problem containers solve	49
Concept	49
How containers solve it	49
5.2 VMs vs Containers	49
Concept	49
The key difference	50
5.3 How containers actually work	50
Linux features	50
A container is just a process	51
5.4 Docker concepts	51
The Docker workflow	51
Terminology	51
5.5 Dockerfile anatomy	52
Concept	52
Our backend’s Dockerfile, annotated	52

Why multi-stage?	52
Common Dockerfile instructions	52
5.6 Docker commands you'll use daily	53
Building	53
Running	53
Inspecting	53
Cleanup	53
5.7 Image layers and caching	53
How caching works	53
Best practice: order layers by frequency of change	54
5.8 Image tags and digests	54
Tags	54
Digests	54
Our lab's approach	54
5.9 Container registries	55
Concept	55
Major registries	55
Authentication	55
5.10 Security best practices	55
Concept	55
5.11 Interview talking points	56
5.12 Exercises	56
Chapter 6: Kubernetes Deep Dive	57
6.1 What Kubernetes is	57
Concept	57
Why orchestration?	57
6.2 The architecture	57
Control plane vs nodes	57
Control plane components	58
Worker node components	58
In AKS	58
6.3 Core objects	58
Pod	59
Deployment	59
How a Deployment works	60
Service	60
ConfigMap and Secret	61
Namespace	61
6.4 kubectl daily commands	61
Getting things	61
Describing (events at the bottom)	62
Logs	62
Writing	62
Debugging	62
Scaling and rollout	62
Events (debugging gold)	62
6.5 Pod lifecycle	62

States	62
Within Running	63
6.6 Pod scheduling	63
How the scheduler picks a node	63
Resource requests vs limits	63
Our lab’s Istio gotcha	64
6.7 Networking inside Kubernetes	64
How traffic flows	64
CNI plugin options	64
6.8 StatefulSet vs Deployment	64
Concept	64
Comparison	65
When to use which	65
6.9 PersistentVolume and PersistentVolumeClaim	65
Concept	65
The relationship	65
PVC YAML	66
Access modes	66
6.10 RBAC	66
Concept	66
Example: Jenkins SA can do helm upgrade	66
6.11 Common pod failure modes	67
Debug workflow	67
6.12 Interview talking points	67
6.13 Exercises	68
Chapter 7: Terraform and Infrastructure as Code	69
7.1 Concept	69
7.2 Why it matters	69
7.3 The core workflow	69
7.4 State — the brain of Terraform	70
Local vs remote state	70
State has secrets	70
In our lab	70
7.5 Building blocks	71
Providers	71
Resources	71
Variables	71
Outputs	71
Data sources (read existing things)	71
Locals (DRY)	72
7.6 Modules — reusable Terraform	72
7.7 Gotchas	72
7.8 In our lab	73
7.9 Interview talking points	73
7.10 Exercises	73
Chapter 8: Helm — Kubernetes Package Manager	75

8.1 Concept	75
8.2 Why it matters	75
8.3 The pieces	76
Chart.yaml	76
values.yaml — defaults	76
templates/deployment.yaml — Go templating	76
Values override per environment	76
8.4 The lifecycle	76
Common commands	77
8.5 Umbrella charts and subcharts	77
8.6 Gotchas	77
8.7 Helm vs Kustomize	78
8.8 In our lab	78
8.9 Interview talking points	78
8.10 Exercises	78
Chapter 9: Azure Specifics	80
9.1 The Azure mental model	80
9.2 Identity — Service Principals and Managed Identities	80
Our lab	81
9.3 Networking — what’s different from AWS	81
NSG gotcha (the one that bit us)	81
9.4 AKS — the managed Kubernetes you’ll see	81
What’s managed for you	81
What you manage	81
Network model choice	82
Node pool gotcha	82
9.5 ACR — Azure Container Registry	82
Auth flows	82
Tag strategy	82
In our lab	82
9.6 Azure RBAC — the rabbit hole	82
Built-in roles you’ll see	83
The gotcha (saved us many hours)	83
9.7 Resource Manager — every API call goes here	83
9.8 Gotchas (a partial list)	83
9.9 In our lab	84
9.10 Interview talking points	84
9.11 Exercises	84
Part II — The Lab, End to End	85
Chapter 10: Lab Architecture Overview	86
10.1 The 30-second pitch	86
10.2 The big picture	86
10.3 Components inventory	87
10.4 The repository layout	87
10.5 What you can demo	88

10.6 Costs	88
10.7 What this lab is <i>not</i>	89
Chapter 11: Iteration 1 — Bootstrap and Network	90
11.1 Goal of this iteration	90
11.2 What we create	90
11.3 Why this order	90
11.4 The bootstrap	91
11.5 The live config	91
11.6 NSG rules — get them right early	92
11.7 Verifying	92
11.8 What iteration 1 <i>doesn't</i> have	92
Chapter 12: Iteration 2 — ACR and AKS	93
12.1 Goal	93
12.2 What we create	93
12.3 The AKS module	93
12.4 Attaching ACR to AKS	94
12.5 Getting credentials	94
12.6 Smoke test	94
12.7 What iteration 2 <i>doesn't</i> have	95
Chapter 13: Iteration 3 — Platform Components	96
13.1 Goal	96
13.2 The order matters	96
13.3 Istio (ingress only mode)	96
13.4 ArgoCD	97
13.5 Argo Rollouts	98
13.6 kube-prometheus-stack	98
13.7 Jenkins (in-cluster)	98
13.8 Verifying	99
13.9 What iteration 3 <i>doesn't</i> have yet	99
Chapter 14: Iteration 4 — Three-Tier App (Helm)	100
14.1 Goal	100
14.2 The shape	100
14.3 Chart layout	100
14.4 Values pattern	101
14.5 Frontend template (without rollouts yet)	102
14.6 Database secret pattern	102
14.7 Installing via ArgoCD	103
14.8 Verifying end-to-end	103
14.9 What iteration 4 <i>doesn't</i> have	103
Chapter 15: Iteration 5 — Real Code + Jenkins Pipeline	104
15.1 Goal	104
15.2 Why kaniko	104
15.3 The Jenkinsfile shape	104

15.4 Gotchas we hit	106
15.5 Why a Git SHA tag and not latest	106
15.6 Manual trigger or every commit?	107
15.7 The end-to-end flow	107
15.8 What we still don't have	107
Chapter 16: Iteration 6 — GitHub Actions + Argo Rollouts + Prometheus Gate	108
16.1 Goal	108
16.2 GitHub Actions pipeline	108
Things we hit and fixed	110
16.3 Argo Rollouts blue/green	110
What “blue/green” means here	110
16.4 AnalysisTemplate — the gate	111
Why two metrics	112
16.5 Auto-promote dev, manual-promote prod	112
16.6 Triggering a failure on purpose (to verify the gate)	112
16.7 What iteration 6 <i>doesn't</i> have	113
Chapter 17: Iteration 7 — CircleCI	114
17.1 Goal	114
17.2 CircleCI vocabulary in 60 seconds	114
17.3 The config	114
17.4 Contexts (where secrets live)	116
17.5 The approval gate	117
17.6 Gotchas we hit	117
17.7 What you can demo	117
17.8 Cross-CI table	117
17.9 What iteration 7 <i>doesn't</i> have	118
Part III — Advanced Topics	119
Chapter 18: Observability	120
18.1 Concept — three pillars (and a fourth)	120
18.2 Why it matters	120
18.3 The Prometheus model	120
18.4 Key metric types	121
18.5 The RED method and the USE method	121
18.6 Istio + Prometheus in our lab	121
18.7 Logs — what we don't have (yet)	122
18.8 Tracing — also not in the lab	122
18.9 SLOs and error budgets	122
18.10 Interview talking points	122
18.11 Exercises	123
Chapter 19: Security	124
19.1 Defense in depth	124
19.2 The least-privilege rule	124
19.3 Secrets — the eternally-difficult problem	124

Kubernetes Secret is base64, not encryption	125
What we do in the lab	125
19.4 mTLS and zero trust	125
19.5 Image security	125
19.6 NetworkPolicy	126
19.7 Pod Security Standards	126
19.8 Supply chain — the hot topic	126
19.9 In our lab (security summary)	127
19.10 Interview talking points	127
19.11 Exercises	127
Chapter 20: Production-Grade Gaps	129
20.1 What this lab is not	129
20.2 High availability	129
20.3 Storage	129
20.4 Security	129
20.5 Observability	130
20.6 Deployment	130
20.7 Compliance	130
20.8 Cost	130
20.9 How to discuss the gaps in an interview	131
Chapter 21: Interview Questions and Answers	132
21.1 Cloud and Azure	132
21.2 Kubernetes	132
21.3 GitOps and CI/CD	133
21.4 Networking	133
21.5 Terraform	134
21.6 Observability	134
21.7 Security	134
21.8 Behavioural / system-design adjacent	135
Appendix A: Cheatsheets	136
A.1 kubectl	136
A.2 helm	137
A.3 terraform	137
A.4 az	137
A.5 docker	137
A.6 PromQL one-liners	138
Appendix B: Building PDF and DOCX	139
Appendix C: Where to Go Next	140

Preface

What this handbook is

A complete journey from “what is the cloud” to “blue/green deployment with Prometheus gates.”

Built around a real working lab on Azure — an AKS cluster with a three-tier app, three CI/CD pipelines, GitOps, progressive delivery, and observability. Every concept is grounded in something you can run.

The lab repo is at <https://github.com/gkhandale-aziro/azure-platform-lab>.

How this handbook is structured

The handbook has four parts:

- **Part I — Foundations** (Chapters 1-9). Cloud, Linux, Git, Networking, Containers, Kubernetes, Terraform, Helm, Azure. Reads top-to-bottom for beginners; skip what you already know.
- **Part II — Building the Lab** (Chapters 10-17). The seven iterations, with every line explained.
- **Part III — Advanced Topics** (Chapters 18-21). Observability, Security, Production gaps, Interview Q&A.
- **Part IV — Appendix** (A, B, C). Cheatsheets, PDF generation, where to go next.

How each chapter is structured

Every chapter follows the same pattern:

Concept

What the thing is. Plain English first, technical definition second.

Why it matters

The motivation. What problem it solves.

How it works

The mechanics. Diagrams, code, examples.

Gotchas

The things that trip people up.

In our lab

How this concept shows up in the lab we're building.

Interview talking points

How to discuss this in a technical interview.

Exercises

Hands-on practice at the end of every chapter.

How to use this handbook

1. **Read each chapter slowly.** Don't try to memorize. Build understanding.
 2. **Run every command.** Type, don't copy-paste. Muscle memory matters.
 3. **Use the companion Glossary.** When you hit an unfamiliar term, look it up there.
 4. **Do the exercises.** They're where learning actually happens.
 5. **Compare to what you already know.** "How is this like X I've seen before?"
 6. **Take notes in your own words.** Don't highlight. Write your own summary.
-

Part I — Foundations

Chapter 1: The Cloud and Why We're Here

1.1 What the cloud actually is

Concept

The cloud is just other people's computers. Specifically, massive data centers full of physical servers, with software that lets you rent slices of them by the hour or second.

You make an API call. Sixty seconds later, you have a Linux VM. You stop using it. The hourly charges stop.

Why it matters

Before the cloud, getting a server meant:

1. Order hardware from a vendor (weeks)
2. Wait for delivery
3. Drive to a colocation facility
4. Rack and cable the server
5. Install operating system
6. Configure networking
7. Hope nothing breaks

The cloud reduces this to:

```
az vm create -g myrg -n myvm --image Ubuntu2204 --size Standard_B2s
```

Two minutes. Done. No hardware purchase.

This shift transformed how we build software. Infrastructure becomes a tool you reach for, not a project you commit to.

How it works

Cloud providers operate at massive scale. A single Azure region contains 3+ data centers, each holding 100,000+ physical servers. Software (the **hypervisor**) carves each server into many virtual machines.

You request a VM via API. The cloud provider's scheduler finds capacity, provisions your VM on shared hardware, gives you a public IP and an OS image. You pay for the time you use it.

Service models

Cloud services come in three layers, each managing more of the stack for you:

IaaS - you manage from OS up

Your app	← you manage
Runtime (Node, Python, etc.)	← you manage
OS (Linux, Windows)	← you manage
Virtualization	← cloud manages
Hardware	← cloud manages

Examples: Azure VM, AWS EC2

PaaS - you only manage your app

Your app	← you manage
Runtime, OS, virtualization, hardware	← cloud manages

Examples: Azure App Service, Azure Kubernetes Service, Heroku, Google App Engine

SaaS - you don't manage anything

Your data	← you manage
The entire stack	← cloud manages

Examples: Office 365, GitHub, Salesforce

Picking the right model

The right model depends on tradeoffs:

Model	More flexibility	Less work	More lock-in
IaaS			Low
PaaS			Medium
SaaS			High

Rule of thumb: pick the highest layer that solves your problem. Don't run a VM if a PaaS will

do. Don't write your own auth system if Office 365 SSO works.

In our lab

We use a mix:

- **IaaS:** AKS worker nodes (we manage the OS, kubelet runs there)
- **PaaS:** AKS control plane (Azure runs etcd, API server, scheduler), ACR (container registry)
- **SaaS:** GitHub, CircleCI (we just use them; they run themselves)

The boundary between “we manage” and “Azure manages” is what makes this a real platform engineering problem.

Gotchas

Cost spirals. A forgotten VM is \$80/month. A misconfigured Storage Account exporting terabytes is thousands. Always tag resources, set budgets, review costs weekly.

Lock-in. Higher service layers mean more lock-in. PaaS often uses proprietary APIs (DynamoDB Cosmos DB Cloud Spanner). SaaS contracts can be hard to exit.

Free Trial limits. Don't assume Free Trial = production-grade. Free SKUs have different rate limits, feature sets, and SLAs.

Interview talking points

Q: “Why pick PaaS over IaaS for Kubernetes?”

“We picked AKS (managed Kubernetes, a PaaS) instead of self-managed Kubernetes on VMs because we want the cloud handling control plane upgrades and certificate rotation, not us. The tradeoff is reduced flexibility — we can't tweak etcd config or run alternative API servers — but for a typical workload that's a feature, not a limitation.”

Q: “What's the cost story here?”

“Our lab runs ~\$140/month at 24/7. By using `az aks stop` overnight and on weekends, it drops to ~\$35/month. Control plane is free in the Standard SKU; we only pay for the worker node VMs. For production we'd use the Premium SKU for the SLA, which adds about \$75/month.”

Exercises

1. Create a free Azure account (<https://azure.microsoft.com/free>) — gets you \$200 of credit for 30 days.
2. Install the Azure CLI on your laptop.
3. Run `az login` and identify your subscription ID.
4. Create a resource group:

```
az group create -n test-rg -l eastus2
```
5. List groups:

```
az group list --output table
```

6. Delete the test group:

```
az group delete -n test-rg --yes
```

7. Read the pricing page for one Azure service (<https://azure.microsoft.com/en-us/pricing/>). Notice cost dimensions.

1.2 The three big clouds

Concept

There are three major cloud providers globally. Each has its strengths, its loyal customer base, and its own naming for similar services.

The contenders

Aspect	Azure	AWS	Google Cloud (GCP)
Owner	Microsoft	Amazon	Google
2026 market share	~24%	~30%	~12%
Started	2010	2006	2008 (publicly available 2011)
Strongest in	Enterprise, Windows shops, .NET, hybrid cloud	Pure breadth, mature service catalog	Data, ML, Kubernetes (they invented it)
Identity	Microsoft Entra ID (formerly Azure AD)	IAM	Cloud IAM
Compute	Azure Compute, AKS	EC2, EKS	Compute Engine, GKE
Object storage	Azure Blob	S3	Cloud Storage
Managed K8s	AKS	EKS	GKE
Container registry	ACR	ECR	Artifact Registry
CDN	Azure Front Door	CloudFront	Cloud CDN

Picking a cloud

Most companies pick a cloud for business reasons, not technical ones:

- **Existing Microsoft licenses** → Azure
- **Existing Amazon retail relationships** → AWS
- **Existing Google Workspace** → GCP
- **Multi-cloud strategy (FinOps optimization)** → all three

Technically, all three are mature. Picking one over another for “better Kubernetes” is rarely a strong argument.

Why this book uses Azure

Three reasons:

1. **Free Trial credit** — \$200 of credit for 30 days, easier than AWS Free Tier
2. **Enterprise context** — most India-based platform engineering roles work with Azure
3. **All concepts transfer** — what you learn here applies to AWS/GCP with renamed services

Interview talking points

Q: “When would you choose AKS over EKS or GKE?”

“Mostly business alignment. AKS for shops already on Azure — Entra ID SSO, ExpressRoute on-prem, enterprise agreements. EKS for mature AWS shops with deep service ecosystem usage. GKE for Google-aligned shops or teams that want Autopilot mode for zero node management. Technically all three are mature. AKS has the best Workload Identity story today. EKS has the broadest compute options (Fargate, multiple AMI lineages). GKE invented Kubernetes and historically leads on autoscaling and Autopilot.”

1.3 Regions and availability zones

Concept

Cloud providers run their hardware in **regions** — geographic areas, each with one or more data centers.

Within a region, **availability zones** (AZs) are physically separated data centers with independent power, cooling, and network.

Why this matters

Three reasons to care about regions:

1. **Latency** — closer to users = faster requests
2. **Data residency** — laws like GDPR (Europe), DPDP (India) require data stays in country
3. **Service availability** — not every Azure service is in every region

Three reasons to care about AZs:

1. **Fault tolerance** — one data center can fail (fire, network outage); other AZs in the region stay up
2. **Latency** — same-AZ is faster than cross-AZ
3. **Cost** — cross-AZ traffic has small per-GB charges that add up at scale

How regions and AZs are organized

Region: East US 2

AZ 1	AZ 2	AZ 3
Data	Data	Data
Center A	Center B	Data
	Center C	

Low-latency private fiber between AZs

For high availability, spread workloads across all three AZs. Single-AZ workloads are vulnerable to AZ-level outages.

Common Azure regions

Region	Location	Notes
eastus	Virginia, USA	Cheapest US region, all services
eastus2	Virginia, USA	Used by our lab, all services
westus2	Washington, USA	Paired with eastus2 for HA
centralindia	Pune, India	India data residency
southeastasia	Singapore	Asia primary
westeurope	Netherlands	Europe primary

Resource groups

A **resource group** is a logical container for related resources. Like a folder.

Azure Subscription

```

Resource Group: myapp-rg-dev
  VM
  Storage Account
  Disk
Resource Group: myapp-rg-prod
  VMs (3)
  Storage Accounts (2)
  Disks (3)
Resource Group: shared-rg
  VNet
  Log Analytics

```

Everything in a resource group can be deleted with one command:

```
az group delete -n myapp-rg-dev --yes
```

Resources can only be in **one** resource group. You can reference resources across groups (the VNet in `shared-rg` can host VMs from `myapp-rg-prod`), but each is owned by one group.

In our lab

Subscription: Free Trial

Resource Group: gskplat-rg-tfstate

```
Storage Account (Terraform state)
Resource Group: gskplat-rg-platform
  ACR (container registry)
  AKS cluster
  VNet + subnets + NSGs
  Log Analytics Workspace
```

Two RGs:

1. `tfstate` for the Terraform state backend
2. `platform` for the actual workload

Cost model

Cloud is rented. You pay per hour (or per second). Common cost dimensions:

Resource	What costs money
VMs	vCPU + RAM × hours, plus disk, plus outbound bandwidth
Storage	GB × month, plus transactions
Networking	Outbound bandwidth (egress) and inter-region traffic
Managed services	Usually free control plane; you pay for underlying VMs

Lab cost reality:

- Our 2-node D2s_v3 cluster runs about \$140/month at 24/7
- `az aks stop` overnight drops it to ~\$35/month
- Control plane is free in AKS Standard SKU
- ACR Basic is \$5/month

Interview talking points

Q: “How do you design for high availability in Azure?”

“First, spread workloads across all three AZs in a region. For AKS, use a node pool with `zones = ['1', '2', '3']`. For databases, use a multi-AZ managed service. Second, replicate to a paired region for disaster recovery. Azure has built-in paired regions like eastus2/westus2. Third, use Azure Front Door or Traffic Manager for global load balancing. Fourth, ensure data has appropriate replication — GRS for cold data, RA-GRS for read replicas across regions.”

Exercises

1. Look up the Azure region map: <https://azure.microsoft.com/global-infrastructure/geographies/>
2. Check which services are in your nearest region:

```
az provider list --query "[].{Namespace:namespace}" -o table
```

3. Create a resource group in a specific region.

4. List AZs available in a region:

```
az vm list-skus --location eastus2 --resource-type virtualMachines --output table | head -
```

Chapter 2: Linux for Platform Engineers

2.1 Why Linux

Concept

You will live in Linux terminals. Every Kubernetes cluster runs on Linux nodes. Every Docker image is based on a Linux distribution. Every CI runner you'll use is Linux. SSH into a remote server: Linux.

Master the basics, or struggle forever.

What you need to know

This chapter covers what platform engineers use daily. Not exhaustive — pragmatic.

By the end of this chapter you'll be able to:

- Navigate the filesystem efficiently
 - Read and search through files and logs
 - Manage processes
 - Connect to remote servers via SSH
 - Set up shell aliases for productivity
 - Understand permissions
 - Use systemd for service management
-

2.2 The shell

Concept

A **shell** is the program that reads your typed commands and runs them. The black/green text terminal you see is the shell.

Common shells:

- **bash** — Bourne Again Shell. Default on most Linux. Portable.
- **zsh** — Z Shell. Default on macOS. More features, plugin ecosystem (Oh My Zsh).
- **fish** — Friendly Interactive Shell. Newer, more user-friendly defaults.

For platform engineering, learn **bash** for scripts (portable across systems) and use **zsh** for daily interactive work.

Key shell features

The shell has powerful features for chaining commands.

Pipes (|) — send the output of one command into the next:

```
ps aux | grep node | awk '{print $2}'
```

This lists all processes, filters for “node”, extracts the second column (PID). Three small tools chained.

Redirection (>, >>, <) — read from or write to files:

```
ls > files.txt           # write output to file (overwrite)
ls >> files.txt           # append output to file
cat < files.txt           # read input from file
```

Background processes (&) — run a command in the background:

```
kubectl port-forward svc/web 8080:80 &
```

The shell returns control to you immediately; the process keeps running.

Variables (\$VAR) — store values:

```
NAME=alice
echo "Hello, $NAME"
```

Command substitution (\$(...)) — use a command’s output as a value:

```
TODAY=$(date +%Y-%m-%d)
echo "Today is $TODAY"
```

Useful shell tips

Tab completion — start typing, press Tab. Shell completes the command/path if unambiguous.

History — press Up arrow to recall previous commands. **Ctrl+R** searches history.

Aliases — give long commands short names in ~/.bashrc:

```
alias k=kubectl
alias gs='git status'
alias kgp='kubectl get pods'
```

After adding aliases, **source ~/.bashrc** or open a new terminal.

2.3 Essential commands

File navigation

Command	What it does
<code>ls</code>	List files in current directory
<code>ls -la</code>	List with details, including hidden files
<code>cd /path</code>	Change directory
<code>cd ..</code>	Go up one directory
<code>cd -</code>	Go to previous directory
<code>pwd</code>	Print working directory
<code>tree</code>	Show directory tree (install with <code>apt install tree</code>)
<code>find . -name "*.yaml"</code>	Recursively search for YAML files

Reading files

Command	What it does
<code>cat file.txt</code>	Print whole file
<code>head -20 file.txt</code>	First 20 lines
<code>tail -20 file.txt</code>	Last 20 lines
<code>tail -f file.log</code>	Follow new log lines (useful for live logs)
<code>less file.txt</code>	Paginated viewer (press <code>q</code> to quit)

Searching files

Command	What it does
<code>grep "error" file.log</code>	Find lines containing “error”
<code>grep -i "error" file.log</code>	Case-insensitive
<code>grep -r "TODO" .</code>	Recursive search in current directory
<code>grep -v "DEBUG" file.log</code>	Lines NOT matching
<code>grep -n "error" file.log</code>	Show line numbers

Text processing

Command	What it does
<code>awk '{print \$1}' file</code>	Print first column
<code>awk '{print \$NF}' file</code>	Print last column
<code>sed 's/old/new/g' file</code>	Substitute “old” with “new” everywhere
<code>cut -d',' -f2 file.csv</code>	Extract column 2 from CSV
<code>sort</code>	Sort lines alphabetically
<code>sort -n</code>	Sort numerically
<code>uniq -c</code>	Count unique lines (input must be sorted)
<code>jq '.field'</code>	Parse JSON (install with <code>apt install jq</code>)

Processes

Command	What it does
<code>ps aux</code>	List all running processes
<code>ps aux grep node</code>	Find processes matching “node”
<code>pgrep -f "pattern"</code>	Find PIDs matching pattern
<code>kill PID</code>	Send SIGTERM (graceful)
<code>kill -9 PID</code>	Send SIGKILL (force)
<code>top</code>	Live process viewer (press q to quit)
<code>htop</code>	Better top (install with <code>apt install htop</code>)

Network

Command	What it does
<code>ss -tlnp</code>	Show listening TCP ports
<code>ss -tan</code>	Show all TCP connections
<code>lsof -i :8080</code>	What process owns port 8080
<code>curl -v https://example.com</code>	Verbose HTTP request
<code>dig example.com</code>	DNS query
<code>ping host</code>	Test reachability
<code>traceroute host</code>	Show network path

File transfer

Command	What it does
<code>scp file user@host:/path</code>	Copy file to remote host
<code>scp user@host:/path file</code>	Copy file from remote host
<code>rsync -avz src/ user@host:/dst/</code>	Sync directory (efficient)
<code>curl -O https://example.com/file</code>	Download (preserve filename)
<code>wget https://example.com/file</code>	Download (alternative)

2.4 Permissions

Concept

Every Linux file has owner, group, and “others” permissions. They look like this:

```
-rwxr-xr-x 1 alice staff 4096 Jan 15 12:00 myfile
```

The first column is the permission string:

```
-  rwx  r-x  r-x
↑   ↑   ↑   ↑
    "others" can read+execute
    "group" can read+execute
    "owner" can read+write+execute
file type (- = file, d = directory, l = symlink)
```

Numeric representation

Each set of `rwX` translates to a number:

```
r = 4
w = 2
x = 1

rwx = 7
r-x = 5
r-- = 4
--- = 0
```

So `chmod 755` means `rwXr-Xr-X` (common for scripts).

`chmod 600` means `rw-----` (common for SSH keys — only owner can read).

Common commands

```
chmod 755 file           # rwxr-xr-x
chmod +x script.sh      # add execute permission
chown alice file        # change owner to alice
chown alice:staff file  # change owner and group
sudo                   # run as root
```

SSH key permissions

SSH is strict about key permissions:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
chmod 600 ~/.ssh/authorized_keys
```

If permissions are too open, SSH refuses to use the keys.

2.5 SSH and remote work

Concept

SSH (Secure Shell) is how you connect to remote servers. Encrypted, key-based authentication is the standard.

Generate a key pair

```
ssh-keygen -t ed25519 -C "your@email.com"
```

This creates:

- `~/.ssh/id_ed25519` — private key (NEVER share, NEVER commit)
- `~/.ssh/id_ed25519.pub` — public key (safe to share)

Put the public key on remote servers in `~/.ssh/authorized_keys` to enable login.

SSH config file

Stop typing `ssh user@host -p 2222 -i ~/.ssh/special_key` every time. Use `~/.ssh/config`:

```
Host azurelab
  HostName 172.30.44.145
  User aziro
  Port 22
  ServerAliveInterval 60
  ServerAliveCountMax 3
  IdentityFile ~/.ssh/azure_lab_key

Host github.com
  User git
  IdentityFile ~/.ssh/github_ed25519
```

Now just `ssh azurelab` instead of the long form.

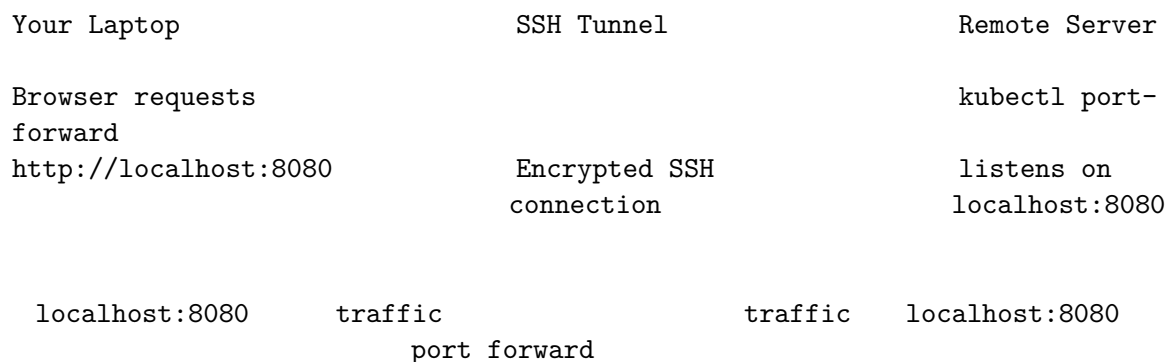
LocalForward (port tunneling)

A killer SSH feature. The `LocalForward` directive forwards a local port through the SSH connection to a remote port.

```
Host azurelab
  HostName 172.30.44.145
  User aziro
  LocalForward 8080 localhost:8080
  LocalForward 9090 localhost:9090
```

Now when you `ssh azurelab`, opening `http://localhost:8080` on your laptop reaches `localhost:8080` on the remote server.

How LocalForward works



We use this constantly in the lab to access Jenkins, ArgoCD, Grafana — services running inside the AKS cluster — from a laptop browser.

Useful SSH tips

Run a one-off command:

```
ssh azurelab 'kubectl get pods -A'
```

Copy a file:

```
scp myfile.txt azurelab:/home/aziro/  
scp azurelab:/home/aziro/myfile.txt .
```

Keep connections alive when idle:

```
Host *  
    ServerAliveInterval 60  
    ServerAliveCountMax 3
```

Add to top of ~/.ssh/config. SSH sends a keep-alive every 60 seconds.

2.6 Environment variables

Concept

Environment variables are key-value strings available to processes. Common uses:

- Configuration (AZURE_SUBSCRIPTION_ID)
- Secrets in CI (API_KEY)
- Shell behavior (PATH, HOME, EDITOR)

Setting variables

```
MY_VAR=value           # set for current shell only  
export MY_VAR=value    # set for current shell AND child processes  
echo $MY_VAR           # print value  
echo "${MY_VAR}_suffix" # use in strings  
unset MY_VAR           # remove  
env                    # list all environment variables
```

The difference between MY_VAR=value and export MY_VAR=value is critical:

- Without export, only the current shell can see it
- With export, child processes (including commands you run) inherit it

Persisting across logins

To make variables available every time you log in, add to ~/.bashrc (bash) or ~/.zshrc (zsh):

```
export AZURE_SUBSCRIPTION="my-sub-id"  
export PATH="$PATH:/usr/local/myapp/bin"
```

For sensitive values, use a separate file with restricted permissions:

```
# ~/.azure-lab.env
export ARM_CLIENT_ID="..."
export ARM_CLIENT_SECRET="..."
export ARM_TENANT_ID="..."
export ARM_SUBSCRIPTION_ID="..."

chmod 600 ~/.azure-lab.env

# At the end of ~/.bashrc:
[ -f ~/.azure-lab.env ] && source ~/.azure-lab.env
```

Now every new shell auto-sources the secrets file. Our lab uses this pattern.

2.7 systemd

Concept

systemd manages long-running services on modern Linux. It's PID 1 (the first process the kernel starts). Everything else runs under it.

Key commands

```
systemctl status nginx      # is nginx running?
sudo systemctl start nginx  # start
sudo systemctl stop nginx   # stop
sudo systemctl restart nginx # restart
sudo systemctl enable nginx # start at boot
sudo systemctl disable nginx # don't start at boot
journalctl -u nginx         # see service logs
journalctl -fu nginx        # follow service logs
journalctl --since "1 hour ago" # logs from last hour
```

When you'll use systemd

You won't deploy systemd services in Kubernetes (containers handle their own supervision). But you'll:

- Diagnose systemd-managed services on VMs (sshd, kubelet, containerd)
- Read service logs to figure out why something failed at boot
- Sometimes write your own systemd unit files for cron-like tasks

Service status meanings

active (running)	- service is up and running
inactive (dead)	- not running
failed	- crashed; check logs
~ activating	- starting up
~ deactivating	- shutting down

If you see `failed`, immediately run `journalctl -u <service>` to see what went wrong.

2.8 Exercises

1. SSH into a VM (AWS Lightsail or Azure VM, both have free tiers).
 2. Generate an SSH key pair if you don't have one.
 3. Add an SSH config alias for your VM.
 4. Set up a LocalForward in your SSH config and verify it works.
 5. Use `tail -f` on a log file while doing something that updates it (e.g., `tail -f /var/log/syslog` while you `sudo ls /` in another terminal).
 6. Use `grep`, `awk`, and pipes to extract specific data from `ps aux`.
 7. Create an alias for your most common `kubectl` command and source it.
 8. Read `systemd` logs for the SSH service: `journalctl -u ssh --since "1 hour ago"`.
-

Chapter 3: Git and Version Control

3.1 What Git is

Concept

Git tracks **snapshots** of your project over time. Each snapshot is a **commit** containing:

- A unique hash (SHA, like `abc1234567...`)
- An author and timestamp
- A message describing what changed
- A reference to the parent commit(s)

Commits form a directed acyclic graph (DAG). A **branch** is a moveable pointer to a commit. **HEAD** is a pointer to the current commit.

Why Git matters

Version control is non-negotiable for any non-trivial work:

- Experiment freely; revert mistakes
- Collaborate without overwriting each other's work
- Audit who changed what, when, why
- Roll back to any past state
- Branch off to try ideas without disturbing main

Git is the de-facto standard. Every team uses it. Every CI system pulls from Git.

3.2 The three states of a file

Concept

A file in a Git repository moves through three states:

Working directory

```
git add
```

Staging area

```
git commit
```

Repository (history)

A file can be:

- **Untracked** — Git doesn't know about it
- **Modified** — tracked, with uncommitted changes
- **Staged** — modified, ready to commit
- **Committed** — saved in history

Visualizing state transitions

Edit file git add git commit

Working dir → Staging → Repository
Modified Staged Committed

git checkout
(restore from history)

3.3 Daily commands

Setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global pull.rebase true          # cleaner history
```

Starting a repo

```
git init                                     # start tracking current directory
git clone https://github.com/user/repo.git   # clone an existing repo
```

Checking status and changes

```
git status                                # what's changed?
git diff                                  # show changes (unstaged)
git diff --staged                         # show staged changes
git log --oneline -20                     # recent commits
git log --graph --all                     # visual history
```

Making changes

```
git add file.txt                           # stage one file
git add .                                  # stage everything
git commit -m "fix: typo"                  # commit staged changes
git commit -am "fix: typo"                 # add + commit (only tracked files)
```

Working with branches

```
git branch                # list branches
git branch feature/x      # create branch (don't switch to it)
git checkout feature/x    # switch to branch
git checkout -b feature/x # create + switch
git checkout master       # switch back
git merge feature/x       # merge branch into current
git branch -d feature/x   # delete branch
```

Synchronizing with remote

```
git remote -v            # list remotes
git fetch                 # download remote refs (no merge)
git pull                  # fetch + merge
git pull --rebase         # fetch + rebase (cleaner)
git push                  # upload current branch
git push origin master    # explicit remote + branch
```

3.4 Branching strategies

Concept

How a team organizes branches determines how they collaborate. Three common patterns.

Git Flow (heavy, old-fashioned)

master	(production)
develop	(next release)
feature/foo	(work branches)
release/v1.2	(stabilization)
hotfix/x	(emergency fixes)

Lots of branches, lots of ceremony. Rarely used now.

GitHub Flow (modern, recommended)

master	(always deployable)
feature/a	(merged via PR)

feature/b (in progress)

- **master** (or **main**) is always deployable
- **feature/*** branches are short-lived
- Open a PR → review → merge → delete branch

This is what most modern teams use. Our lab uses this.

Trunk-Based Development

Everyone commits to **master** directly. Heavy automated testing. Feature flags hide incomplete features.

Used by Google, Meta. Requires mature CI/CD and culture.

3.5 Pull Requests

Concept

A **Pull Request** (PR) — also called Merge Request in GitLab — is a web UI workflow for proposing changes from one branch into another. It's not part of Git itself; GitHub/GitLab add this layer.

A PR page contains:

- The diff (what changed)
- CI status (tests pass/fail)
- Comments on specific lines
- Review approvals
- Merge controls

Why PRs matter

PRs are the gate between work and production:

- **Code review** catches bugs and improves quality
- **CI** catches regressions automatically
- **Approvals** create accountability
- **Audit trail** for compliance

A team without PRs deploys whatever anyone wrote. A team with PRs has guardrails.

Anatomy of a good PR

Title: Brief, imperative mood ("Add user search" not "Added user search")

Description: ## Summary
 What this PR does in 2-3 sentences.

 ## Why
 Why this change is needed. Link to ticket if applicable.

```

    ## How to test
    Step-by-step verification.

    ## Risks
    What might go wrong. Migration concerns.

Diff size:    Small. 300-400 lines of real code max.
              Big PRs get bad reviews.

```

3.6 Merge conflicts

Concept

When two branches changed the same line, Git can't decide which version to keep. You decide.

What a conflict looks like

When you `git pull --rebase` or `git merge` and there's a conflict, Git modifies the file with markers:

```

def get_user(user_id):
<<<<<<< HEAD
    # Your version
    return db.users.find_one({"id": user_id})
=====
    # Their version (from the branch being merged)
    return User.objects.get(pk=user_id)
>>>>>>> commit-sha

```

Three options:

1. Keep yours: delete their version + markers
2. Keep theirs: delete your version + markers
3. Combine: write a new version that takes both into account

Resolving conflicts

After conflict during pull/merge/rebase:

```

git status                                # see conflicting files
# Edit each conflicted file. Remove <<<<<<<, =====, >>>>>>> markers.

git add file.py                          # mark as resolved
git diff --name-only --diff-filter=U     # any remaining conflicts?

git rebase --continue                    # if rebasing
git merge --continue                     # if merging

```

```
# To abort and try again:
git rebase --abort
git merge --abort
```

or

Tips for fewer conflicts

- Keep branches short-lived (1-3 days max)
 - Pull from master frequently (`git pull --rebase origin master`)
 - Communicate when working in the same area of the code
 - Small PRs merge fast, before conflicts develop
-

3.7 Rewriting history (carefully)

Concept

Git lets you rewrite history with several commands. **All of them are dangerous on shared branches.**

The commands

```
git commit --amend      # change last commit (message or contents)
git reset HEAD~1        # undo last commit, keep changes staged
git reset --hard HEAD~1 # undo last commit, LOSE changes
git rebase -i HEAD~5    # interactive: squash, reorder, edit
git revert <commit>     # create NEW commit that undoes <commit>
```

The cardinal rule

Never rewrite history that's been pushed to a shared branch.

Why: if I have your old commit `abc1234` and you rewrite it to `def5678`, my local repo doesn't know — it sees `abc1234` as missing and refuses to push. We've diverged. Painful to fix.

For shared work, use `git revert` (creates a new commit that undoes the change).

When rewriting is safe

- On your local branches before pushing
- Before opening a PR, to clean up history
- After someone else's review, to address feedback in a single commit

Common pattern: clean up before push

```
# Make 5 commits while developing
git commit -m "wip: try x"
git commit -m "wip: still trying"
git commit -m "wip: finally got it"
git commit -m "fix typo"
git commit -m "fix another typo"
```

```
# Before pushing, squash into one clean commit
git rebase -i HEAD~5
# In the editor, mark commits to "squash" or "fixup"
# Save. Git presents a merged commit message editor.

# Now one clean commit. Push:
git push origin feature/x
```

3.8 .gitignore

Concept

Some files shouldn't be tracked: build outputs, secrets, IDE settings, local logs. A `.gitignore` file in the repo root tells Git what to skip.

Common entries

```
# Editor / IDE
.vscode/
.idea/
*.swp

# Build artifacts
node_modules/
__pycache__/
*.pyc
dist/
build/

# Logs
*.log

# Environment / secrets
.env
.env.*
*.key
*.pem
sp-terraform.json

# Terraform
.terraform/
*.tfstate
*.tfstate.backup
*.tfplan

# OS
```

.DS_Store
Thumbs.db

Critical for security

The `.env` and `*.json` exclusions prevent committing secrets accidentally. Our lab's `.gitignore` excludes `sp-terraform.json` (the file with the Service Principal's password).

3.9 Git in CI/CD

Concept

Every CI/CD system uses Git as its source. The pattern is:

```
git clone <repo>
git checkout <branch-or-sha>
# run tests, build, deploy
```

The `<sha>` is the immutable identifier of a specific commit. Even if branches move, the SHA stays the same.

Why we use short SHA as image tags

In our lab, every Docker image is tagged with the git short SHA:

```
myacr.azurecr.io/myapp:sha-abc1234
```

This means:

- **Immutable:** the tag never changes. Once pushed, that image is fixed.
- **Traceable:** from the image tag, you can find the exact commit, exact diff, exact author.
- **Rollback is trivial:** revert the bump commit in Git; ArgoCD reconciles back to the previous SHA.

A useful workflow

```
# 1. Make changes
git checkout -b feature/x
# ... edit files ...
git commit -am "feat: add new endpoint"

# 2. Push and open PR
git push origin feature/x
# (open PR on GitHub)

# 3. CI builds image
# CI runs: docker build -t myacr.azurecr.io/myapp:sha-$(git rev-parse --short=7 HEAD) .

# 4. Deploy commits the new SHA
# CI runs:
```

```
# - Update values-dev.yaml to point at new SHA
# - git commit -m "ci(dev): bump to sha-abc1234"
# - git push

# 5. ArgoCD detects the commit and applies to cluster.

# To rollback:
git revert <bump-commit-sha>
# ArgoCD reconciles back to previous image.
```

3.10 Interview talking points

Q: “How would you roll back a bad deploy?”

“Three layers depending on how recent the bad deploy is. Within the rolling window — Argo Rollouts keeps the previous ReplicaSet alive for 30 seconds after a promote, so `kubectl argo rollouts undo` is instant. Beyond that window, `git revert <bump-commit>` and let ArgoCD reconcile — image tags are git SHAs, so reverting the commit means deploying the previous image. The longest path: forensics, identify which commit introduced the regression, revert that specific change, redeploy. Every deploy is a Git commit, so the audit trail makes this easy.”

Q: “What’s your branching strategy?”

“GitHub Flow. Short-lived feature branches off master, PR to merge, CI must be green and one approver. Master is always deployable. We don’t use a develop branch — over-engineering for our team size. For larger releases, we use feature flags rather than long-lived release branches.”

Q: “What if two developers want to work on the same feature?”

“Two patterns. Either we split the feature into independent vertical slices that can be merged independently, or we pair-program on the same branch. Long-running shared branches accumulate merge conflicts and slow everyone down. Short PRs that land daily are much easier to coordinate.”

3.11 Exercises

1. Create a private repository on GitHub.
2. Clone it locally, make a commit, push it.
3. Create a feature branch, change something, push the branch.
4. Open a PR on GitHub, review it (with yourself), merge it.
5. Practice resolving a merge conflict — intentionally create one by editing the same line in two branches.

6. Use `git log --oneline --graph --all` to visualize branch history.
7. Try `git rebase -i HEAD~3` to squash 3 commits into 1.
8. Configure your `.gitconfig` with useful aliases:

```
git config --global alias.lg "log --oneline --graph --all"
git config --global alias.st "status"
git config --global alias.co "checkout"
git config --global alias.br "branch"
```

9. Read your colleagues' PRs on a real project (or any open-source project on GitHub).
-

Chapter 4: Networking Essentials

4.1 Why networking matters

Concept

You cannot operate distributed systems without understanding networking. Cloud infrastructure is networking. Kubernetes is networking. Service meshes are networking.

This chapter covers the layer-by-layer model and the concepts you'll actually use daily.

4.2 The OSI model (simplified)

Concept

The OSI model is a 7-layer abstraction of how data moves between computers. In practice, platform engineers talk about layers 3, 4, and 7.

Layer 7: Application	(HTTP, gRPC, SSH)	← Where you code
Layer 6: Presentation	(TLS encryption)	
Layer 5: Session		
Layer 4: Transport	(TCP, UDP)	← Ports live here
Layer 3: Network	(IP, routing)	← IP addresses
Layer 2: Data Link	(Ethernet)	← MAC addresses
Layer 1: Physical	(cables, radio)	

The L4/L7 distinction comes up constantly:

- **L4 load balancer** — operates on TCP/UDP, no understanding of HTTP
- **L7 load balancer** — understands HTTP, can route by URL, header, etc.

Azure Load Balancer is L4. Application Gateway is L7. Istio's ingress gateway is L7.

4.3 IP addresses and CIDR

Concept

An **IPv4 address** is 32 bits, usually written as four decimal octets: 10.0.1.15.

CIDR notation (Classless Inter-Domain Routing) compactly describes a range: 10.0.0.0/16 means the first 16 bits are fixed, the last 16 bits are available for hosts = 65,536 addresses.

How CIDR sizing works

The number after / is the prefix length. Larger /N = smaller range:

CIDR	Bits free	Addresses	Common use
/8	24	16,777,216	Internet ranges
/16	16	65,536	VNet
/24	8	256	Subnet
/27	5	32	Small subnet (mgmt)
/30	2	4	Point-to-point
/32	0	1	Single host

In Azure, AWS, GCP, you'll usually work in /16 for VNets and /24 or /27 for subnets.

Our lab's networking

VNet: gskplat-vnet-shared	10.0.0.0/16	(65,536 IPs total)
snet-aks	10.0.1.0/24	(256 IPs - cluster nodes)
snet-apps	10.0.2.0/24	(256 IPs - future apps)
snet-mgmt	10.0.3.0/27	(32 IPs - jumpbox/mgmt)

The /16 VNet has room for ~256 /24 subnets. We use 3.

4.4 Public vs Private IPs

Concept

Private IP ranges (RFC 1918) are non-routable on the public internet. They're for internal networks:

10.0.0.0/8	Common in cloud VNets
172.16.0.0/12	Docker default
192.168.0.0/16	Home routers

Everything else is public — globally routable.

How private IPs reach the internet

A VM in a private subnet can't be reached from the internet (no route). To reach the internet outbound, it needs:

- **NAT Gateway** (cloud-managed) — translates many private IPs to a few public IPs
- **Public IP directly attached** to the VM

- **Through a Load Balancer** with a public IP

For inbound traffic from the internet, you need:

- A public IP attached to the resource
- A Load Balancer or App Gateway in front

4.5 DNS

Concept

DNS (Domain Name System) maps human-friendly names to IP addresses.

You type: `api.example.com`
 DNS returns: `20.94.18.66`
 Your browser: connects to `20.94.18.66`

Common record types

Type	Maps to	Example
A	IPv4 address	<code>example.com</code> → <code>1.2.3.4</code>
AAAA	IPv6 address	<code>example.com</code> → <code>2001:db8::1</code>
CNAME	Another domain	<code>www.example.com</code> → <code>example.com</code>
MX	Mail server	<code>example.com</code> → <code>mail.example.com</code> (priority 10)
TXT	Text records (verification)	<code>_acme-</code> <code>challenge.example.com</code> → <code>"abc123"</code>

DNS in Kubernetes

Every Kubernetes cluster runs **CoreDNS** internally. It resolves Service names to ClusterIPs:

`backend.dev.svc.cluster.local`

ClusterIP: `172.16.140.23`

Format: `<service>.<namespace>.svc.cluster.local`

Pods in the same namespace can use the short form `backend`. Cross-namespace requires the namespace prefix: `backend.prod`.

4.6 TCP and UDP, ports

Concept

A **port** is a 16-bit number (0-65535) identifying a specific application on a host. Combined with the IP address, it uniquely identifies an endpoint.

TCP vs UDP

TCP	UDP
Connection-oriented	Connectionless
Reliable, ordered	Best-effort, unordered
3-way handshake	Just send
Slower (more overhead)	Faster (less overhead)
Used by:	Used by:
HTTP/HTTPS	DNS queries
SSH	Streaming media
Databases	VPN
git	Game traffic

Common ports to memorize

Port	Protocol	Service
22	TCP	SSH
80	TCP	HTTP
443	TCP	HTTPS
3306	TCP	MySQL
5432	TCP	PostgreSQL
5678	TCP	(our backend)
6379	TCP	Redis
6443	TCP	Kubernetes API server
9090	TCP	Prometheus
53	UDP	DNS

Ports below 1024 require root to bind. Use 1024+ for user-mode apps.

4.7 Firewalls and NSGs

Concept

A **firewall** filters network traffic based on rules. Each rule has:

- Source IP/range
- Destination IP/range

- Port
- Protocol (TCP/UDP)
- Action (Allow/Deny)
- Priority

Azure's **Network Security Group** (NSG) is a stateful firewall attached to subnets or NICs. "Stateful" means return traffic for an allowed connection is automatically allowed.

Azure's default rules

When you create an NSG, Azure includes these implicit rules at priority 65000+:

Priority	Name	Source	Dest	Port	Action
65000	AllowVnetInBound	VirtualNet	*	*	Allow
65001	AllowAzureLoadBalancerInBound	AzureLB	*	*	Allow
65500	DenyAllInBound	*	*	*	Deny

These mean: cluster-internal traffic is allowed; Azure load balancer health probes are allowed; everything else is denied.

The explicit deny gotcha

If you add your own explicit rule at a LOWER priority number (higher priority), it fires FIRST and can shadow the implicit allows.

Our lab broke in iter 4 because of this. We added:

```
security_rule {
  name      = "DenyAllInboundExplicit"
  priority  = 4000                # ← lower number = higher priority
  access    = "Deny"
  # ... source/dest *, port *, etc ...
}
```

This explicit Deny at priority 4000 fired BEFORE the implicit AllowVnetInBound at 65000. Result: pod-to-pod traffic across nodes died.

The fix: explicit allows BEFORE the explicit deny.

```
security_rule {
  name      = "AllowVnetInbound"
  priority  = 1000                # higher priority than the deny
  access    = "Allow"
  source_address_prefix = "VirtualNetwork"
}
```

```
security_rule {
  name      = "AllowAzureLoadBalancerInbound"
  priority  = 1100
  access    = "Allow"
  source_address_prefix = "AzureLoadBalancer"
```

```

}

security_rule {
  name      = "DenyAllInboundExplicit"
  priority  = 4000          # fires AFTER the allows
  access    = "Deny"
}

```

How NSG rules are evaluated

Incoming packet

Check rules in priority order (lowest number first):

priority 1000	AllowVnetInbound	→ MATCH → ALLOW (stop here)
priority 1100	AllowAzureLBInbound	→ no match, continue
priority 4000	DenyAllInboundExplicit	→ no match (allow already won)
priority 65000	AllowVnetInBound	→ (Azure implicit, never reached)

Packet allowed

The first matching rule wins. Lower priority numbers are evaluated first.

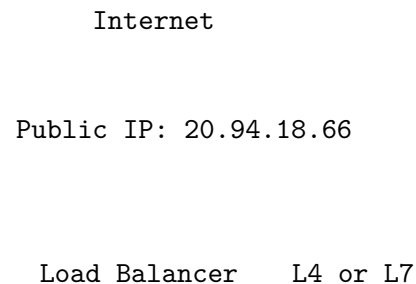
4.8 Load Balancers

Concept

A load balancer distributes incoming connections across multiple backend instances. The two flavors:

- **L4 LB** — distributes TCP/UDP. Doesn't understand HTTP. Fast.
- **L7 LB** — understands HTTP. Can route by URL path, header, host. Adds features like TLS termination, sticky sessions, retries.

Visual



Backend	Backend	Backend
Pod 1	Pod 2	Pod 3

The LB has the public IP. Backend pods have private IPs the LB knows about.

In our lab

We have a public LB (Azure Standard Load Balancer) at 20.94.18.66. It points at the Istio ingress gateway, which then routes L7 to internal Services based on Host header and URL path.

4.9 TLS and HTTPS

Concept

TLS (Transport Layer Security) provides:

- **Encryption** — eavesdroppers can't read traffic
- **Authentication** — verify the server is who it claims
- **Integrity** — detect tampering

HTTPS is HTTP over TLS.

The TLS handshake

Client	Server
ClientHello (supported ciphers, version)	
ServerHello + Certificate	
Verify cert against trusted CA list	
Generate session key, encrypt with server's public key	
Encrypted application data	

Certificates

A certificate proves the server's identity. It's signed by a Certificate Authority (CA) that the client trusts.

For public sites, free CAs like Let's Encrypt issue certs after verifying you control the domain (DNS or HTTP challenge).

Tools to manage certs in Kubernetes:

- **cert-manager** — issues and auto-renews certs from any ACME-compatible CA
- **Azure Key Vault** integration — store certs in KV, mount in pods

mTLS (mutual TLS)

In normal TLS, the server proves its identity, the client doesn't. In **mutual TLS** (mTLS), both sides present certificates and verify each other.

Why: service-to-service inside the cluster. The frontend pod proving it's the frontend pod, not a malicious replacement. Istio does this by default.

Two modes in Istio:

- **PERMISSIVE** — accepts both plain TCP and mTLS. Used during sidecar rollout.
- **STRICT** — accepts only mTLS. Production target.

Our lab uses PERMISSIVE because the Postgres pod opts out of sidecar injection (and would be unreachable with STRICT).

4.10 Interview talking points

Q: “Walk me through what happens when a user hits `dev.lvh.me:59999` in their browser.”

“Public DNS for `dev.lvh.me` resolves to `127.0.0.1` (`lvh.me` does that for any subdomain). Browser connects to `127.0.0.1:59999`. SSH LocalForward in their SSH config catches that, tunnels through SSH to the VM, where `kubectl port-forward` is listening on `9999:80`. That forwards to the Istio ingress Service, which forwards to the istio-ingress pod (Envoy). Envoy sees the Host header `dev.lvh.me`, matches the dev VirtualService, routes to the frontend or backend Service depending on URL path. Service forwards to a pod, which has an istio-proxy sidecar handling mTLS. The sidecar forwards to the app container on localhost. Response goes back through the same chain. Ten hops, encrypted at multiple layers.”

Q: “Why did adding a Deny rule break a working cluster?”

“Azure has implicit Allow rules at priority 65000 for VNet-internal traffic and 65001 for load balancer health probes. Adding an explicit Deny at a lower priority number — say 4000 — fires first because lower number means higher priority. The Deny shadows the implicit allows. Pod-to-pod traffic across nodes dies. Health probes fail. The fix: explicit Allow rules at priority 1000 and 1100 for VNet and AzureLoadBalancer respectively, BEFORE the explicit Deny.”

4.11 Exercises

1. Use `ping`, `traceroute`, `dig` against common domains. See the IPs.
2. `nc -zv google.com 443` — test if a port is open.
3. `curl -v https://example.com` — see the TLS handshake details.
4. Calculate available IPs for `/24`, `/27`, `/30`. Write them out.
5. Read your laptop's routing table: `route -n` (Linux) or `route print` (Windows).
6. Use `ss -tlnp` to see what's listening on TCP on your machine.

Chapter 5: Containers and Docker

5.1 The problem containers solve

Concept

Before containers, deploying an app meant:

1. Install OS
2. Install language runtime (Python, Node, etc.)
3. Install dependencies (system packages, libraries)
4. Copy your code
5. Configure environment variables
6. Start the process
7. Repeat on every server

Result:

- “Works on my machine” — your dev env doesn’t match prod
- Hours wasted on environment drift
- Painful onboarding (new dev = days of setup)
- Hard to reproduce production locally

How containers solve it

A **container** packages the app + all its dependencies (runtime, libraries, system packages, env vars) into a portable image. Run the image anywhere → identical behavior.

Build once: `docker build -t myapp:1.0 .`

Run anywhere: `docker run myapp:1.0`

`docker run myacr.azurecr.io/myapp:1.0`

`kubectl create deployment ... --image=myacr.azurecr.io/myapp:1.0`

Same bytes everywhere.

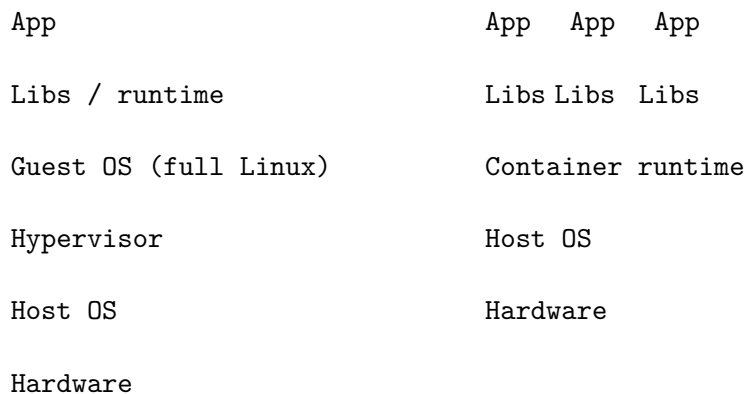
5.2 VMs vs Containers

Concept

VMs and containers both provide isolation, but at different levels.

VM model

Container model



VM size: 1-10 GB	Container size: 5-500 MB
VM boot: 30-120 seconds	Container start: 0.1-2 seconds
VM isolation: hardware	Container isolation: kernel namespaces

The key difference

A VM has a **separate kernel**. A container shares the host kernel and uses kernel features (namespaces, cgroups) for isolation.

This makes containers:

- **Faster** to start (no kernel boot)
- **Smaller** in size (no OS to ship)
- **Lighter** in resource use
- **Less isolated** (kernel exploits could compromise containers)

For most workloads, the speed and density benefits outweigh the slightly weaker isolation.

5.3 How containers actually work

Linux features

Containers use three Linux kernel features:

Namespaces — isolate what processes can see. Six types:

Namespace	Isolates
<code>pid</code>	Process IDs (container sees its own PID 1)
<code>net</code>	Network interfaces (each container has its own)
<code>mnt</code>	Mount points (filesystem view)
<code>user</code>	User and group IDs
<code>ipc</code>	Inter-process communication
<code>uts</code>	Hostname

Cgroups (control groups) — limit resource usage:

- CPU (shares, quotas)
- Memory (limit, OOM behavior)
- I/O (bandwidth, IOPS)
- Network bandwidth

Union filesystems — layer filesystem changes efficiently. The container sees a unified view; the underlying storage is layered.

A container is just a process

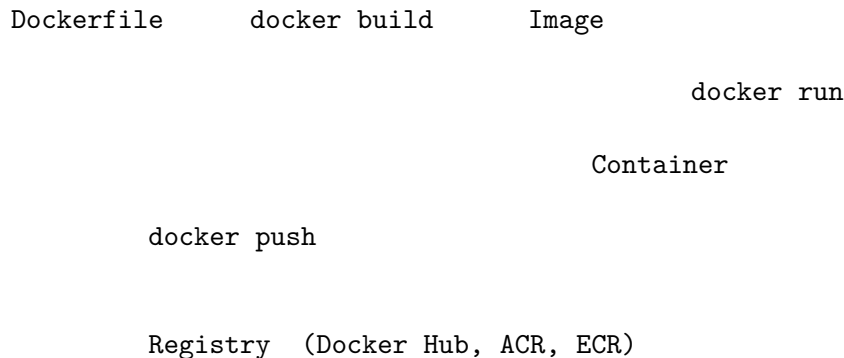
There's no "container daemon" running. A container is a process (or group of processes) isolated by the above kernel features.

`ps -ef` *# see the container processes on the host*

The container runtime (containerd, CRI-O, Docker) just sets up the namespaces and cgroups, then starts the process.

5.4 Docker concepts

The Docker workflow



Terminology

Term	Definition
Image	A read-only template (filesystem layers + metadata)
Container	A running instance of an image
Dockerfile	Instructions to build an image
Registry	A storage service for images
Tag	A human label for an image, like <code>nginx:1.28-alpine</code>
Layer	One step in an image build (each Dockerfile line is a layer)
Digest	A SHA-256 hash uniquely identifying an image's contents

5.5 Dockerfile anatomy

Concept

A Dockerfile is a text file with instructions to build an image. Each instruction creates a layer.

Our backend's Dockerfile, annotated

```
# Stage 1: install dependencies in builder
FROM node:20-alpine3.22 AS deps          # base image (small Linux + Node 20)
WORKDIR /app                             # working directory for commands
COPY package.json package-lock.json ./   # copy deps manifests
RUN npm ci --omit=dev --no-audit          # install prod-only deps

# Stage 2: minimal runtime image
FROM node:20-alpine3.22 AS runtime
WORKDIR /app
USER node                                # run as non-root user

COPY --chown=node:node --from=deps /app/node_modules ./node_modules
COPY --chown=node:node server.js ./

ENV NODE_ENV=production
ENV PORT=5678
EXPOSE 5678                             # documentation: this port is for HTTP

CMD ["node", "server.js"]                # what runs when container starts
```

Why multi-stage?

The final image doesn't include build tools, dev dependencies, or the build cache. Smaller image = faster pull, smaller attack surface, fewer CVEs.

Single stage:	base + build tools + source + node_modules + app	~1 GB
Multi stage:	base + node_modules + app	~150 MB

Common Dockerfile instructions

Instruction	Purpose
FROM	Base image
WORKDIR	Set working directory
COPY	Copy files from build context
RUN	Execute command during build
ENV	Set environment variable
EXPOSE	Document which port the app listens on (informational)
USER	Switch to a different user
CMD	Default command to run when container starts
ENTRYPOINT	Wrap CMD with a fixed prefix
HEALTHCHECK	Define a health check command

Instruction	Purpose
LABEL	Add metadata

5.6 Docker commands you'll use daily

Building

```
docker build -t myapp:1.0 .           # build from Dockerfile in current dir
docker build -t myapp:1.0 -f Dockerfile.prod # use a specific Dockerfile
docker build --target deps -t myapp:deps . # build only the deps stage
docker build --no-cache .             # ignore cache, rebuild everything
```

Running

```
docker run nginx:alpine                # run interactively (foreground)
docker run -d nginx:alpine              # detached (background)
docker run -p 8080:80 nginx:alpine      # expose port 80 inside as 8080 outside
docker run -e MY_VAR=value nginx        # set env var
docker run -v /host/path:/container/path nginx # mount volume
docker run --name web -d nginx          # name the container
```

Inspecting

```
docker ps                               # running containers
docker ps -a                             # all (including stopped)
docker logs web                           # see container output
docker logs -f web                        # follow logs
docker exec -it web sh                    # shell into running container
docker inspect web                        # full metadata
docker stats                              # live resource use
```

Cleanup

```
docker stop web
docker rm web                            # delete stopped container
docker rmi nginx:alpine                  # delete image
docker system prune                      # delete unused stopped containers, networks
docker system prune -a                  # also delete unused images
```

5.7 Image layers and caching

How caching works

Each Dockerfile instruction creates a layer. Docker caches layers. If line N didn't change, layers N are reused.

```
FROM node:20-alpine                    # layer 1 - rarely changes
COPY package.json ./                  # layer 2 - changes when deps change
```

```
RUN npm install           # layer 3 - changes when layer 2 changed
COPY . .                  # layer 4 - changes on every code edit
RUN npm run build         # layer 5 - changes when layer 4 changed
```

If you edit `server.js`:

- Layer 4 changes (new code)
- Layer 5 changes (rebuild)
- Layers 1-3 reuse cache

Build is fast because `npm install` is cached.

Best practice: order layers by frequency of change

Bottom: rarely changes (base image, OS packages)

↓

Top: frequently changes (app code)

This maximizes cache hits and minimizes rebuild time.

5.8 Image tags and digests

Tags

A tag is a human-readable label on an image:

```
nginx                # defaults to docker.io/library/nginx:latest
nginx:1.28-alpine     # specific version
gskplatacrn73d5y.azurecr.io/three-tier/backend:latest  # ACR
gskplatacrn73d5y.azurecr.io/three-tier/backend:sha-46fa90f # immutable
```

Tags are mutable — `nginx:latest` today is different from `nginx:latest` next week.

Digests

A digest is the cryptographic hash of an image's contents. Immutable.

```
nginx@sha256:b3a8c7e5d4f2a1b0c9e8d7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a7b6
```

Two pulls of the same digest are guaranteed identical bytes. Tags can change; digests can't.

Our lab's approach

We tag images by **git short SHA**:

```
myacr.azurecr.io/backend:sha-46fa90f
```

This is mutable (you could re-tag), but in practice we treat it as immutable. The benefits:

- Traceable to the exact commit that produced it
- Same image SHA flows dev → prod (build-once-deploy-many)
- Rollback is trivial: change values.yaml back to the previous SHA

Pure production-grade pinning would use the digest directly. Future improvement.

5.9 Container registries

Concept

A **registry** stores Docker images. Push to push them up; pull to download them.

Major registries

Registry	Use case
Docker Hub	Public images, free tier with rate limits
Azure Container Registry (ACR)	Azure-integrated, works with AKS managed identity
AWS ECR	AWS-integrated, IAM auth
GitHub Container Registry (GHCR)	Free for open source, GitHub Actions integration
Quay.io	Red Hat ecosystem, CVE scanning included
Harbor	Self-hosted, on-prem

Authentication

Docker Hub

```
docker login
```

ACR

```
az acr login -n myacr
```

or with a service principal:

```
docker login myacr.azurecr.io -u $ARM_CLIENT_ID -p $ARM_CLIENT_SECRET
```

Our lab uses ACR with the kubelet identity having `AcrPull` role — so AKS can pull images without storing credentials.

5.10 Security best practices

Concept

Containers don't make you secure. They give you tools to be secure. Common practices:

Practice	Why
Use specific tags, not <code>:latest</code>	Reproducible builds
Run as non-root user (<code>USER node</code>)	Principle of least privilege
Multi-stage builds	Smaller images, fewer CVEs
Scan images with Trivy/Snyk	Catch known vulnerabilities
Use distroless or alpine base	Fewer packages = fewer CVEs
Don't bake secrets into images	Use env vars or secret mounts
Sign images with Cosign	Verify origin at deploy time
Pin base images by digest	Even tag pinning can shift
Drop Linux capabilities	Containers don't need most caps
Set resource limits	Prevent runaway containers

Our lab does the first 5. Image signing, capability dropping, and digest pinning are documented production gaps.

5.11 Interview talking points

Q: “Walk me through a Dockerfile review.”

“I’d look for: specific base image tag (not latest). Multi-stage build to keep final image small. `USER non-root` for principle of least privilege. Layer ordering — dependencies before code for cache efficiency. `.dockerignore` to keep the build context small. `EXPOSE` and `CMD` clearly defined. Red flags: secrets in `ENV`, `chmod 777`, no `HEALTHCHECK`, `apt-get install` without cleaning `/var/lib/apt/lists/`.”

Q: “How do you keep images small?”

“Multi-stage builds. The build stage has the toolchain; the runtime stage has only what’s needed to run. For Node apps that’s `node:alpine` with just `node_modules`. For Go binaries that can be `scratch` — empty base, only the compiled binary. Layer ordering matters too — put rarely-changing stuff at the top so cache hits stay good.”

Q: “Why not just use `:latest` tag?”

“`:latest` is mutable. What it points to changes over time. Reproducibility breaks: a deploy that works today might fail tomorrow because `:latest` updated. For production, use immutable tags — semver, git SHA, or digest. We use git short SHA in our lab so every image traces to a specific commit.”

5.12 Exercises

1. Install Docker locally (or use Docker Desktop).
 2. Write a Dockerfile for a simple Python script.
 3. Build it, run it, expose a port, hit it with curl.
 4. Convert it to multi-stage build.
 5. Run `docker history myapp:1.0` and explain each layer.
 6. Scan it: `docker scout cves myapp:1.0` (or use Trivy).
 7. Push to Docker Hub (free account).
 8. Run two containers from the same image with different env vars.
-

Chapter 6: Kubernetes Deep Dive

6.1 What Kubernetes is

Concept

Kubernetes (k8s) is a **container orchestrator**. You declare what you want (“3 copies of this app, exposed on port 80, restart if any die”). Kubernetes makes it happen and keeps it happening.

Why orchestration?

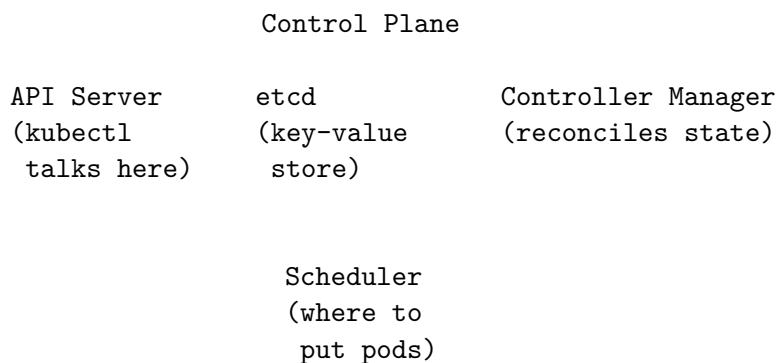
Manual `docker run` doesn’t scale beyond one server:

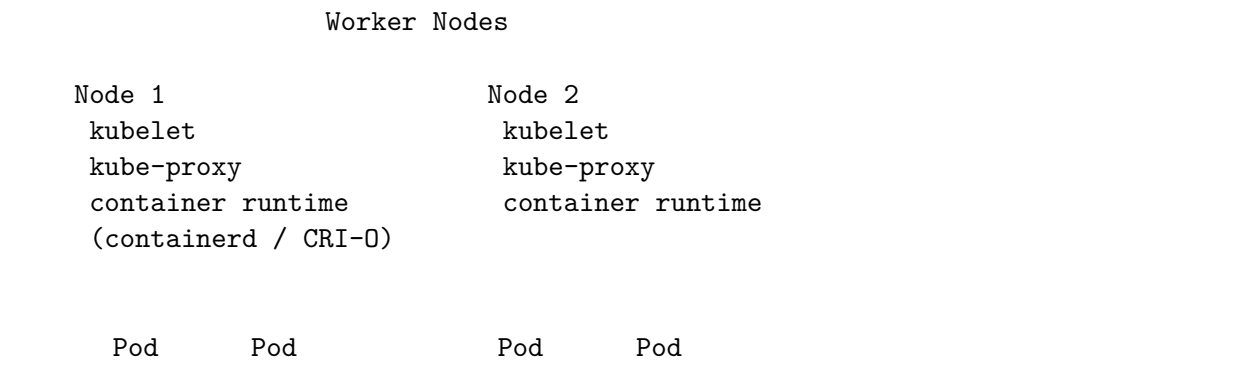
- Need health checks and automatic restarts
- Need scheduling — which server runs which container?
- Need load balancing across container replicas
- Need secrets and config management
- Need networking between containers across hosts
- Need rolling updates without downtime
- Need autoscaling

Kubernetes solves all of these. It’s the de-facto standard.

6.2 The architecture

Control plane vs nodes





Control plane components

Component	Role
API Server	Front door. kubectl talks to this. All state changes go through it.
etcd	Key-value store. Persistent cluster state.
Scheduler	Decides which node runs each Pod.
Controller Manager	Runs control loops (DeploymentController, ReplicaSetController, etc.).

Worker node components

Component	Role
kubelet	Talks to API server. Manages Pod lifecycle on this node.
kube-proxy	Implements Service networking (iptables/IPVS rules).
Container runtime	Runs containers (containerd, CRI-O, Docker).

In AKS

The control plane is managed by Azure (free in Standard SKU, paid in Premium with SLA). You only see and pay for worker nodes.

You can't SSH into the control plane. You interact via the kubectl-friendly API endpoint.

6.3 Core objects

The objects you'll create most often.

Pod

The smallest deployable unit. 1+ containers that share network and storage.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx:1.28-alpine
    ports:
    - containerPort: 80
```

You rarely create Pods directly. Use Deployments or StatefulSets.

Deployment

Manages a set of identical Pods, handles rolling updates.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: nginx
        image: myacr.azurecr.io/frontend:sha-abc123
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 512Mi
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 2
```

```
periodSeconds: 5
```

How a Deployment works

You apply Deployment

↓

Deployment Controller creates a ReplicaSet

↓

ReplicaSet Controller creates Pods to match replicas count

↓

Scheduler picks nodes for the Pods

↓

kubelet on each node pulls images, runs containers

When you update the image:

↓

Deployment creates a NEW ReplicaSet with new image

↓

Old ReplicaSet scales down, new one scales up

↓

After completion, old ReplicaSet kept at 0 replicas (for rollback)

Service

A stable network endpoint for a set of Pods. Pods come and go; the Service stays.

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: ClusterIP           # internal only (default)
  selector:
    app: frontend          # matches Pod labels
  ports:
    - port: 80              # service port
      targetPort: 80        # pod port
```

Service types:

Type	Behavior
ClusterIP	Internal IP only (default)
NodePort	Opens same port on every node's external IP
LoadBalancer	Cloud provider creates external LB pointing at nodes
ExternalName	DNS alias to an external service

ConfigMap and Secret

Inject configuration into Pods.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info
  DB_HOST: postgres.dev.svc.cluster.local

apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
data:
  password: PEJBU0U2NCBFTkNPREVEPg==      # base64
```

Secrets are NOT encrypted by default, just base64-encoded. Use External Secrets + Azure Key Vault for real secrets.

Namespace

Logical isolation within a cluster.

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    istio-injection: enabled      # opt into Istio sidecars
```

Our lab uses: dev, prod, argocd, argo-rollouts, jenkins, istio-system, monitoring.

6.4 kubectl daily commands

Getting things

<code>kubectl get pods</code>	<i># default namespace</i>
<code>kubectl get pods -n dev</code>	<i># specific namespace</i>
<code>kubectl get pods -A</code>	<i># all namespaces</i>
<code>kubectl get pods -o wide</code>	<i># extra columns</i>
<code>kubectl get pods -o yaml</code>	<i># full YAML output</i>
<code>kubectl get pods --watch</code>	<i># live updates</i>
<code>kubectl get pods -l app=frontend</code>	<i># filter by label</i>

Describing (events at the bottom)

```
kubectl describe pod my-pod -n dev      # pod events and details
kubectl describe node aks-node-1        # node info
```

Logs

```
kubectl logs my-pod                    # logs from main container
kubectl logs my-pod -c sidecar          # specific container
kubectl logs my-pod --previous          # from last instance (if it crashed)
kubectl logs -f my-pod                  # follow live
kubectl logs -l app=frontend --tail=50 # last 50 lines from all matching pods
```

Writing

```
kubectl apply -f manifest.yaml          # create or update
kubectl delete -f manifest.yaml         # delete from manifest
kubectl edit deployment frontend        # edit live (avoid in prod)
```

Debugging

```
kubectl exec -it my-pod -- sh           # shell into running pod
kubectl port-forward svc/frontend 8080:80 # forward local port
kubectl run debug --rm -it --image=busybox -- sh # ephemeral debug pod
kubectl cp my-pod:/tmp/file ./file      # copy file out of pod
```

Scaling and rollout

```
kubectl scale deployment frontend --replicas=5
kubectl rollout status deployment/frontend
kubectl rollout history deployment/frontend
kubectl rollout undo deployment/frontend    # rollback
```

Events (debugging gold)

```
kubectl get events -A --sort-by='.lastTimestamp' | tail -20
```

6.5 Pod lifecycle

States

Pending

(scheduler decides where to place)

Running

Succeeded Failed

(gone)

Within Running

Containers have probes:

- **Liveness probe** — “is the app alive?” Restart on failure.
- **Readiness probe** — “is the app ready for traffic?” Remove from Service LB until ready.
- **Startup probe** — “has it started yet?” Grace period before liveness kicks in.

```
spec:
  containers:
  - name: backend
    readinessProbe:
      httpGet:
        path: /health
        port: 5678
      initialDelaySeconds: 2
      periodSeconds: 5
```

6.6 Pod scheduling

How the scheduler picks a node

The scheduler considers:

- **Resource requests** — does the node have free CPU/memory?
- **Node selector / nodeAffinity** — does the node have required labels?
- **Tolerations / taints** — can the pod tolerate the node’s taints?
- **Pod affinity / anti-affinity** — should it be near or far from other pods?

Resource requests vs limits

```
spec:
  containers:
  - name: app
    resources:
      requests:
        cpu: 100m           # 0.1 CPU
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 512Mi
```

	What it means
requests	Minimum reserved for the pod. Scheduler uses this to decide placement.

	What it means
limits	Maximum the container can use. Hit the CPU limit → throttled. Hit memory limit → OOM kill.

Our lab's Istio gotcha

Default Istio sidecar requests were 100m CPU each. With 8 pods $\times$ 100m = 800m just for sidecars. On a 2 vCPU node with system pods taking 1100m, the scheduler couldn't fit them all.

Fix: trim sidecars via annotations:

```
metadata:
  annotations:
    sidecar.istio.io/proxyCPU: "10m"
    sidecar.istio.io/proxyMemory: "64Mi"
```

Saves ~270m CPU per namespace.

6.7 Networking inside Kubernetes

How traffic flows

Pod-to-Pod (same node)	→ via veth pair + Linux bridge
Pod-to-Pod (across nodes)	→ via overlay network (kubenet, Calico, Cilium)
Pod-to-Service	→ kube-proxy DNAT to a Pod IP
Pod-to-Internet	→ via node IP (SNAT)
External-to-Service	→ via LoadBalancer or Ingress

CNI plugin options

CNI	What
kubenet (Azure default, our lab)	Simple, NAT-based, no direct pod-to-pod across cluster boundaries
Azure CNI	Pods get VNet IPs, route to other Azure resources
Calico	Open source, network policies, eBPF mode
Cilium	eBPF-based, very fast, observability features

Production typically uses Azure CNI Overlay or Cilium. Our lab uses kubenet (Free Trial-friendly).

6.8 StatefulSet vs Deployment

Concept

Most apps are stateless — replace a pod with another, no big deal. Some apps have identity — databases, brokers, leader-elected systems. StatefulSet handles these.

Comparison

Deployment:	StatefulSet:
Pods are cattle	Pods are pets
pod-7c8d-xyz1	mydb-0
pod-7c8d-xyz2	mydb-1
pod-7c8d-xyz3	mydb-2
Random names	↑ ordinal, stable
Random order start	Ordered start (0 → 1 → 2)
Random delete	Reverse delete (2 → 1 → 0)
Shared PVC (if any)	Per-pod PVC
No stable network ID	Stable DNS: mydb-0.mydb.<ns>...

When to use which

Use Deployment	Use StatefulSet
Stateless apps	Databases (Postgres, MySQL)
Web frontends	Message brokers (Kafka, RabbitMQ)
API backends	Distributed consensus (etcd, Zookeeper)
Worker pods	Apps that need stable identity

Our lab: frontend and backend are Rollouts (similar to Deployment). Postgres is a StatefulSet.

6.9 PersistentVolume and PersistentVolumeClaim

Concept

Pods are ephemeral; their filesystems disappear on restart. For data that needs to persist, use **PersistentVolume** (PV) and **PersistentVolumeClaim** (PVC).

The relationship

```
StorageClass    "managed-csi" (Azure disk)
```

```
                (provisions on PVC creation)
```

```
PersistentVolume 10Gi disk in Azure
```

```
                (bound)
```

```
PersistentVolumeClaim "postgres-data"
```

```
                (mounted)
```

```
Pod    /var/lib/postgresql/data
```

PVC YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: managed-csi
  resources:
    requests:
      storage: 1Gi
```

Access modes

Mode	Behavior
ReadWriteOnce (RWO)	One node at a time. Azure Disk, EBS. Standard for databases.
ReadOnlyMany (ROX)	Many nodes can read.
ReadWriteMany (RWX)	Many nodes can write. Azure Files, NFS. Slower.

Our lab uses:

- managed-csi (Azure Disk, RWO) for Postgres
- azurefile-csi (Azure Files, RWX) for npm/trivy caches shared across CI pods

6.10 RBAC

Concept

Role-Based Access Control. Who can do what.

ServiceAccount ← pod identity (what's running)

Role / ClusterRole ← permissions
bound by

RoleBinding / ClusterRoleBinding
granted via

Example: Jenkins SA can do helm upgrade

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-deployer
rules:
```

```

- apiGroups: [""]
  resources: ["pods", "services", "secrets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer-dev
  namespace: dev
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: jenkins
roleRef:
  kind: ClusterRole
  name: jenkins-deployer

```

Rule of thumb: ClusterRole is reusable. RoleBinding namespaces it.

6.11 Common pod failure modes

State	Meaning
Pending	Insufficient resources, node selector mismatch, PVC not bound
Init:0/1	Init container is running
CrashLoopBackOff	Container exits, k8s restarts, exits, ...
ImagePullBackOff	Can't pull the image (wrong tag, no creds, network)
OOMKilled	Used more memory than the limit
Error	Container exit code != 0
Completed	Job done (for one-shot pods)

Debug workflow

```

kubectl describe pod <pod>           # events at the bottom
kubectl logs <pod>                   # what app printed
kubectl logs <pod> --previous         # from last crash
kubectl get events -n <ns> --sort-by='.lastTimestamp'

```

6.12 Interview talking points

Q: “How would you debug a pod stuck in Pending?”

“First `kubectl describe pod <name>` — events at the bottom usually tell you. Common: ‘Insufficient cpu’ — requests too high or cluster full, check `kubectl describe`

`nodes` for allocatable vs requested. ‘FailedScheduling: 0/2 nodes match’ — node selectors, taints, affinity. ‘pvc not bound’ — StorageClass missing or PVC mistyped. ‘No nodes available’ — cluster empty, autoscaler off? Then `kubectl get events -A --sort-by='.lastTimestamp'` for broader context.”

Q: “What’s the difference between requests and limits?”

“Requests are what the scheduler uses to decide placement — the minimum reserved for the pod. Limits are the max the container can use. Hit CPU limit, you get throttled — slow but alive. Hit memory limit, OOM kill — pod restarts. Setting requests too high means failed scheduling. Setting limits too low means OOM kills. Setting them equal is ‘guaranteed’ QoS class — best stability.”

Q: “When would you pick StatefulSet over Deployment?”

“When the workload needs stable identity, stable storage, or ordered startup. Databases — Postgres, MySQL — definitely. Brokers like Kafka, RabbitMQ. Anything with leader election where pod-0 must be reachable as ‘the leader.’ For stateless apps, Deployment is simpler and faster. For our lab Postgres is a StatefulSet, frontend and backend are Rollouts which behave like Deployment.”

6.13 Exercises

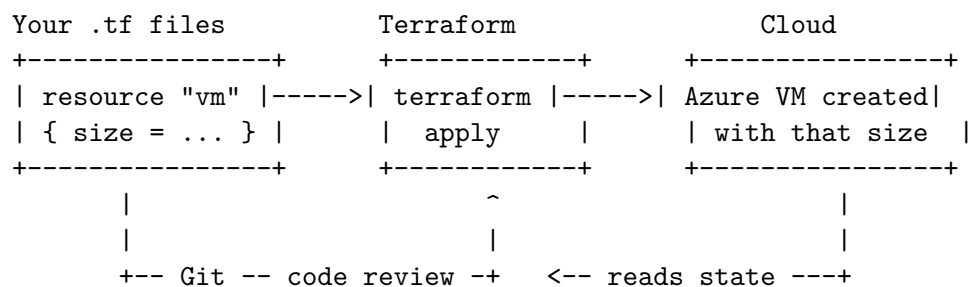
1. Install `kubectl` and configure it for a local cluster (kind, minikube, k3d).
2. Apply a Deployment + Service YAML, expose it, hit it with curl.
3. Create a ConfigMap and mount it in a pod.
4. Trigger an OOM by setting memory limits low and stressing memory.
5. Use `kubectl port-forward` to access an internal service.
6. Write an RBAC policy that lets a ServiceAccount only read pods.
7. Create a StatefulSet with a PVC and verify data persists across pod restart.

Chapter 7: Terraform and Infrastructure as Code

7.1 Concept

Infrastructure as Code (IaC) means your infrastructure — VMs, networks, clusters, DNS records, IAM roles — is defined in text files, version-controlled in Git, and applied through a tool. No more clicking through the Azure portal at 2 AM trying to remember what you set the disk size to.

Terraform is HashiCorp’s IaC tool. You declare the desired state in `.tf` files. Terraform compares it to the actual state, computes a plan (what to add/change/destroy), and applies it.



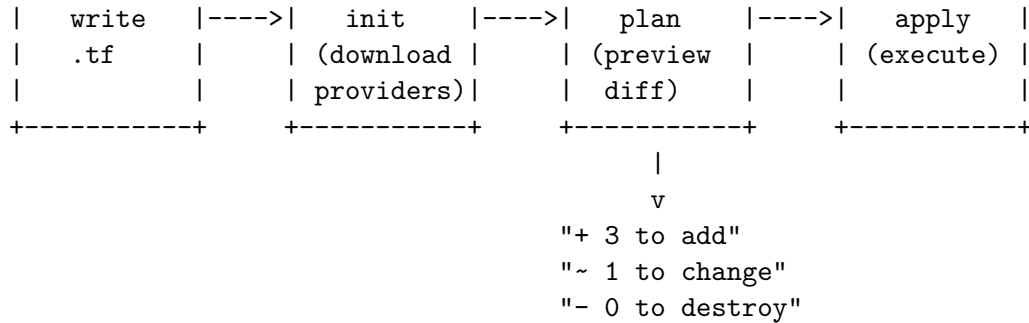
7.2 Why it matters

Without IaC	With Terraform
Click through portal, hope you remember	Code is the source of truth
No review, no diff	PR-reviewed plans before changes
Drift between environments	Same module, different <code>tfvars</code>
Disaster recovery = days	<code>terraform apply</code> and you're back
Onboarding new engineer = “let me show you”	“Read the repo, run <code>plan</code> ”

For a platform engineer, **everything starts here**. The cluster, the networks, the registry, the service principals — all defined in code.

7.3 The core workflow

+-----+	+-----+	+-----+	+-----+
---------	---------	---------	---------



- **terraform init** — downloads providers (Azure, AWS, GCP, Kubernetes, Helm, etc.), initializes the backend.
- **terraform plan** — reads state, compares with code, prints a diff. **Never skip this in real environments.**
- **terraform apply** — applies the plan. Asks for confirmation unless `-auto-approve`.
- **terraform destroy** — tears down everything in the state. Useful for ephemeral labs, terrifying in production.

7.4 State — the brain of Terraform

State is the mapping between your code and what actually exists in the cloud. Terraform stores it in a file called `terraform.tfstate`.

Code says:	State says:	Cloud has:
"resource azurerm_vm vm1"	vm1 -> /subscriptions/.../ vm-7b3c9f	Azure VM with that ID

Local vs remote state

- **Local state** — `terraform.tfstate` in the working directory. Fine for one person, one machine. Lose the file, lose the brain.
- **Remote state** — stored in Azure Blob, S3, or Terraform Cloud. Required for teams. Supports **locking** so two people don't **apply** simultaneously.

State has secrets

Terraform writes resource attributes to state. That includes admin passwords, SP secrets, connection strings. **Treat the state file like a credential.**

In our lab

We use **Azure Blob Storage** for remote state. Bootstrap (`bootstrap/`) creates the storage account, then `live/` Terraform configures itself with backend `"azurerm"` pointing at it. The blob is encrypted at rest, access is controlled by SP role.

7.5 Building blocks

Providers

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}
```

```
provider "azurerm" {
  features {}
}
```

Resources

```
resource "azurerm_resource_group" "rg" {
  name      = "gskplat-prod-rg"
  location = "eastus2"
  tags = {
    env = "prod"
  }
}
```

Variables

```
variable "location" {
  description = "Azure region"
  type        = string
  default     = "eastus2"
}
```

Pass values via .tfvars files:

```
# terraform.tfvars
location = "eastus2"
env      = "prod"
```

Outputs

```
output "rg_name" {
  value = azurerm_resource_group.rg.name
}
```

Data sources (read existing things)

```
data "azurerm_subscription" "current" {}
```

```
output "sub_id" {
  value = data.azurerm_subscription.current.subscription_id
}
```

Locals (DRY)

```
locals {
  common_tags = {
    project = "gskplat"
    owner   = "platform"
  }
}

resource "azurerm_resource_group" "rg" {
  name       = "gskplat-rg"
  location   = var.location
  tags       = local.common_tags
}
```

7.6 Modules — reusable Terraform

A module is a directory of `.tf` files you call from elsewhere.

```
modules/
  aks/
    main.tf
    variables.tf
    outputs.tf
live/
  prod/
    main.tf      <-- calls module "aks"

# live/prod/main.tf
module "aks" {
  source              = "../modules/aks"
  resource_group_name = azurerm_resource_group.rg.name
  node_count          = 2
  vm_size              = "Standard_B2s"
}
```

Rule of thumb: if you'd copy-paste the same block twice, make it a module.

7.7 Gotchas

- **State drift.** Someone clicked in the portal. Terraform now disagrees with reality. Run `terraform refresh` or `terraform plan` to see drift, then either import the change or revert in the portal.
- **destroy is non-reversible.** Always plan `-destroy` first. Verify the resource list.
- **Secrets in state.** Never commit `.tfstate` to Git. Use remote state with encryption.

- **Provider version drift.** Pin providers (~> 4.0). A new major version can change defaults and break things silently.
- **Long applies.** AKS takes 10-15 minutes. `terraform apply` will sit there. Don't kill it — leave it running.
- **Lock file** (`.terraform.lock.hcl`). Commit it. It pins exact provider versions for the team.

7.8 In our lab

We follow the standard layout:

```
infra/
  bootstrap/      # creates the state storage account (run once)
  modules/
    network/      # VNet, subnets, NSGs
    aks/           # AKS cluster + node pool
    acr/           # container registry
    monitoring/    # Log Analytics workspace
  live/
    dev/           # dev environment, calls modules
    prod/          # prod environment, calls modules
```

Each `live/<env>/` has its own state file (in the shared storage account but with different keys), so dev applies can never break prod.

7.9 Interview talking points

Q: “Why Terraform over ARM/Bicep/CloudFormation?”

“Terraform is cloud-agnostic — same tool for Azure, AWS, GCP, Kubernetes, Helm, GitHub. ARM/Bicep are Azure-only. For a multi-cloud or platform-engineering role, Terraform is the lingua franca. Also: huge module ecosystem, mature state management, and `plan` gives you a real diff before applying.”

Q: “How do you handle Terraform state for a team?”

“Remote backend with locking — Azure Blob with the `azurerm` backend, S3 + DynamoDB for AWS. Never local state in a real team. Separate state per environment (dev/staging/prod) so a dev apply can't blow up prod. State has secrets so the bucket/account is locked down with IAM.”

Q: “How do you handle secrets in Terraform?”

“Never in code. Use environment variables (`TF_VAR_*`), CI secrets, or pull from a vault — Azure Key Vault, AWS Secrets Manager, HashiCorp Vault — at runtime via data source. State still has them, so encrypt the state and restrict access.”

7.10 Exercises

1. Install Terraform locally. Run `terraform version`.
2. Write a `.tf` file that creates one Azure Resource Group. `init`, `plan`, `apply`.

3. Add a variable for region. Override it with `-var`, then with a `.tfvars` file.
 4. Move the RG into a module. Call it from a root config.
 5. Configure remote state in Azure Blob. Migrate from local to remote.
 6. Run `terraform destroy`. Recreate. Notice state recreates from scratch.
 7. Import an existing Azure resource into Terraform with `terraform import`.
-

Chapter 8: Helm — Kubernetes Package Manager

8.1 Concept

A Helm **chart** is a parameterized bundle of Kubernetes YAML. Think of it like a Debian package, but for k8s.

```
+-----+
| my-app/ (chart) |
| +-- Chart.yaml | <-- metadata (name, version)
| +-- values.yaml | <-- default config
| +-- templates/ | <-- YAML with {{ .Values.foo }} placeholders
|     +-- deploy.yaml
|     +-- service.yaml
+-----+
```

You install it like this:

```
helm install myapp ./my-app -f values-prod.yaml
```

Helm renders the templates with values, applies the resulting YAML to the cluster, and tracks it as a **release**.

8.2 Why it matters

Without Helm	With Helm
Copy-paste 20 YAML files per env	One chart, many values.yaml
Hand-edit image tag everywhere	One value <code>image.tag</code>
Hard to upgrade	<code>helm upgrade</code> with rollback
No package registry	OCI registries (ACR, Docker Hub)

For platform engineers, Helm is **the** way to deliver internal apps to teams and consume off-the-shelf components (Prometheus, ArgoCD, cert-manager).

8.3 The pieces

Chart.yaml

```
apiVersion: v2
name: three-tier
version: 0.1.0
appVersion: "1.0.0"
description: A web + api + db demo app
```

values.yaml — defaults

```
frontend:
  image:
    repository: nginx
    tag: latest
  replicaCount: 1
```

templates/deployment.yaml — Go templating

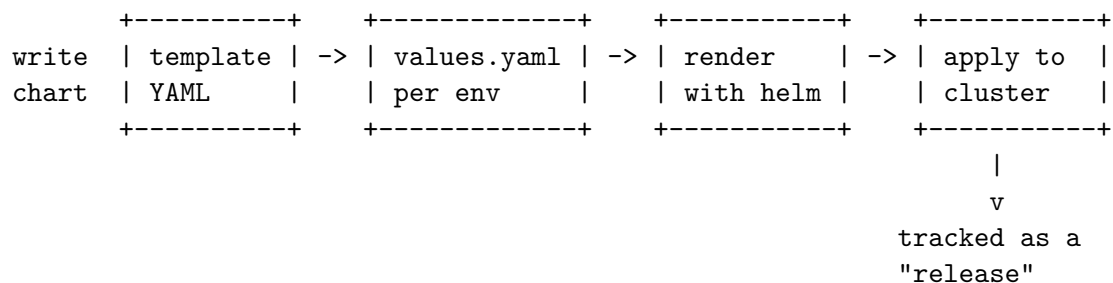
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-frontend
spec:
  replicas: {{ .Values.frontend.replicaCount }}
  template:
    spec:
      containers:
      - name: frontend
        image: "{{ .Values.frontend.image.repository }}:{{ .Values.frontend.image.tag }}"
```

Values override per environment

```
helm install dev ./my-app -f values-dev.yaml
helm install prod ./my-app -f values-prod.yaml
```

Same chart. Two completely different deployments.

8.4 The lifecycle



Common commands

```
helm install <release> <chart>           # first install
helm upgrade <release> <chart>           # change config
helm upgrade --install <release> ...     # idempotent (install or upgrade)
helm rollback <release> <revision>       # back to a prior release
helm list -A                             # all releases everywhere
helm uninstall <release>                 # delete everything
helm template <chart> -f values.yaml     # render without applying (dry-run)
```

8.5 Umbrella charts and subcharts

A chart can depend on other charts. This is how you ship “an app stack” as one unit.

```
three-tier/                                <-- umbrella chart
  Chart.yaml                             (declares dependencies)
  values.yaml                           (parent values)
  charts/                                <-- subcharts live here
    frontend/
    backend/
    database/

# Chart.yaml
dependencies:
- name: frontend
  version: 0.1.0
  repository: file://charts/frontend
- name: backend
  version: 0.1.0
  repository: file://charts/backend
- name: database
  version: 0.1.0
  repository: file://charts/database
```

One `helm install` brings up the whole stack. One `helm uninstall` tears it down.

8.6 Gotchas

- **helm template lies sometimes.** It doesn’t validate against the cluster’s schema. Use `helm install --dry-run` for that.
- **Image tag = latest.** Don’t. Pin to a digest or SHA. `latest` rolls silently and breaks reproducibility.
- **Helm 2 was tiller.** Helm 3 dropped tiller. If you see references to tiller, that’s old.
- **Whitespace in templates.** `{{- }}` strips whitespace. Forget it and you’ll get blank lines that break YAML.
- **Subchart values are namespaced.** To set frontend’s `replicaCount` in the umbrella `values.yaml`, write `frontend.replicaCount` not `replicaCount`.
- **helm upgrade --force is dangerous.** It recreates resources, which means downtime for `Services/StatefulSets`.

8.7 Helm vs Kustomize

	Helm	Kustomize
Approach	Templating (Go templates)	Patching (YAML overlays)
Power	High (loops, conditionals)	Medium (overlays, generators)
Complexity	Higher learning curve	Lower
Built into kubectl	No	Yes (<code>kubectl apply -k</code>)
Best for	Distributable packages	Env-specific tweaks

Many teams use **both**: Helm for upstream things (Prometheus, ArgoCD), Kustomize for in-house overlays.

8.8 In our lab

The whole three-tier app is a Helm umbrella chart at `kubernetes/apps/three-tier/`. We have:

- `values.yaml` — defaults (image tag `latest`, replicas 1)
- `values-dev.yaml` — dev overrides (sha tag, blueGreen auto-promote)
- `values-prod.yaml` — prod overrides (sha tag, blueGreen manual gate)

ArgoCD applications point at these charts. When CI changes `values-dev.yaml` and commits, ArgoCD re-runs `helm template` and applies the diff.

8.9 Interview talking points

Q: “Helm or Kustomize?”

“Depends. Helm for distributing things — packages, internal platforms. The templating + values pattern is the right fit when ‘one chart serves N environments and M teams.’ Kustomize for environment overlays of YAML you own — patch the dev replicas, patch the staging image. Many real systems use both: Helm to install upstream tools, Kustomize for env-specific patches.”

Q: “How do you handle secrets in Helm?”

“Don’t put secrets in `values.yaml`. Either: (1) sealed-secrets / sops-encrypted values committed to git, (2) Helm pulls from a vault at install time, (3) use external-secrets-operator to sync from Key Vault into k8s Secrets. The chart references the Secret by name; the value comes from elsewhere.”

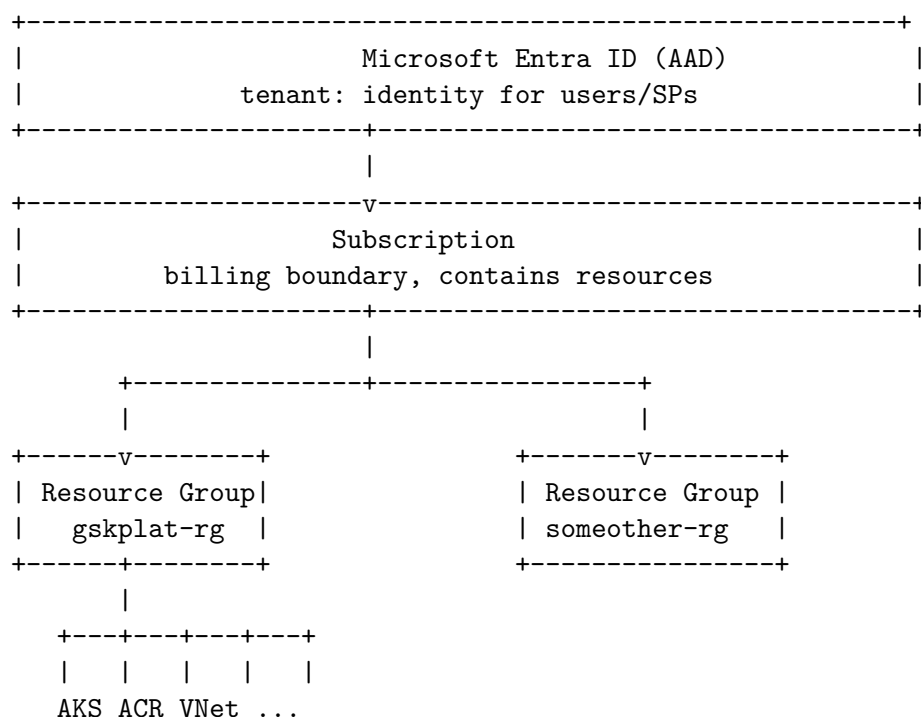
8.10 Exercises

1. `helm create my-chart`. Inspect the generated files.
2. Install it. Use `helm get values` and `helm get manifest` to inspect.
3. Override one value with `-f` and `--set`.
4. `helm upgrade` with a new value. Then `helm rollback`.
5. Add a subchart. Set a subchart value from the umbrella.
6. Run `helm template ... | kubectl apply -f -`. Notice this skips Helm tracking.

7. Push a chart to an OCI registry (ACR supports OCI charts).
-

Chapter 9: Azure Specifics

9.1 The Azure mental model



- **Tenant** — your Microsoft Entra ID directory. Identity lives here.
- **Subscription** — a billing container. Resources are billed to one subscription.
- **Resource Group (RG)** — a logical bag of resources. Delete the RG = delete everything in it.
- **Resource** — anything: VM, VNet, AKS, ACR, storage account.

9.2 Identity — Service Principals and Managed Identities

A **Service Principal (SP)** is a non-human identity used by automation (Terraform, CI). Has a client ID + client secret (or cert).

A **Managed Identity (MI)** is an SP that Azure manages for you — no secret to rotate. Two flavors:

- **System-assigned** — tied to a resource's lifecycle (deleted with the resource).

- **User-assigned** — standalone, can be attached to multiple resources.

Rule of thumb: prefer MI when the workload runs in Azure. Use SP only when something outside Azure (laptop, GitHub Actions) needs to authenticate.

Our lab

Terraform runs from a VM with an SP (**sp-terraform**). The SP has Contributor + User Access Administrator on the subscription scope. Credentials sit in `~/.azure-lab.env` (chmod 600) and `sp-terraform.json` (gitignored).

9.3 Networking — what’s different from AWS

Concept	AWS	Azure
Private network	VPC	Virtual Network (VNet)
Subnet	Subnet	Subnet
Firewall (subnet-level)	NACL	NSG (Network Security Group)
Firewall (instance-level)	Security Group	also NSG
Public DNS	Route 53	Azure DNS
Load Balancer	ELB/ALB/NLB	Azure LB / App Gateway / Front Door
Egress	NAT Gateway	NAT Gateway / Azure Firewall

NSG gotcha (the one that bit us)

Azure has **implicit** rules at priority 65000+: `AllowVnetInBound`, `AllowAzureLoadBalancerInBound`, `DenyAllInBound`. If you write an **explicit** `DenyAllInbound` at high priority, **it overrides the implicit allow rules**. Pod-to-pod traffic across nodes and LB health probes will break.

Fix: either omit the explicit `DenyAll` and trust the implicit one, or pair it with explicit `AllowVnetInBound` and `AllowAzureLoadBalancerInBound` rules at lower priority numbers.

9.4 AKS — the managed Kubernetes you’ll see

What’s managed for you

- The control plane (API server, etcd, scheduler) — Microsoft runs it. You don’t see the master nodes.
- Cluster upgrades — you trigger them, AKS handles the dance.
- Integration with Azure RBAC, AAD, ACR.

What you manage

- Node pools (system + user pools).
- Network model (kubenet vs Azure CNI).
- Workloads, RBAC, ingress.

Network model choice

	kubenet	Azure CNI
Pod IPs	NAT'd, from a separate range	Real VNet IPs
Subnet usage	Light	Heavy (one IP per pod)
Performance	Slower (NAT)	Faster (direct)
Network policies	Limited	Full
Best for	Labs, simple setups	Production, advanced networking

Our lab: kubenet (lighter on IPs, fine for B2s nodes and dev work).

Node pool gotcha

- **System pool** — runs CoreDNS, metrics-server. Don't taint it for workloads.
- **User pool** — your apps. Add as separate node pools so you can scale/upgrade independently.

9.5 ACR — Azure Container Registry

```
+-----+   docker push   +-----+   kubelet pulls   +-----+
| CI/dev | -----> | gskplat...azurecr.io | -----> | AKS |
+-----+               +-----+               +-----+
```

Auth flows

- **From a dev box (you):** `az acr login --name <reg>` (uses your AAD token).
- **From AKS:** attach ACR to AKS — `az aks update --attach-acr` grants the AKS managed identity `AcrPull`.
- **From other CI:** SP with `AcrPush/AcrPull` role.

Tag strategy

- **Avoid latest** in deployments. Use SHA tags (`sha-9b61859`) for immutability.
- **Vulnerability scanning** — ACR has built-in Defender for Containers (paid).
- **Geo-replication** — premium SKU only.

In our lab

`gskplatacrn73d5y.azurecr.io` is attached to AKS. CI builds tag `sha-<git-sha>`, pushes, and updates `values.yaml` to point at the new tag.

9.6 Azure RBAC — the rabbit hole

Azure RBAC = role assignments at scopes.

Scope (where it applies)	Role (what they can do)	Principal (who)
/subscriptions/xxx	Contributor	sp-terraform
/resourceGroups/yyy	Reader	gopal@...

```
| /resourceGroups/yyyy/ | | User Access | | aks-mi |
| providers/.../zzz | | Administrator | | |
+-----+ +-----+ +-----+
```

Built-in roles you'll see

Role	Scope of power
Owner	Full + assign roles
Contributor	Full minus assign roles
Reader	Read-only
User Access Administrator	Only manage role assignments
AcrPull / AcrPush	ACR-specific
AKS Cluster User	Pull kubeconfig

The gotcha (saved us many hours)

A Terraform SP with **only Contributor** will fail when it tries to create role assignments — for example, attaching ACR to AKS. Symptom: `AuthorizationFailed: ... does not have authorization to perform action 'Microsoft.Authorization/roleAssignments/write'.`

Fix: also grant **User Access Administrator** at the subscription scope (or the relevant resource group).

9.7 Resource Manager — every API call goes here

```
+-----+   HTTPS REST   +-----+   +-----+
| az CLI / | -----> | Azure Resource Mgr | -> | Resource |
| Terraform|          | (ARM API endpoint) |   | providers|
+-----+   +-----+   +-----+
```

Everything in Azure goes through ARM. The `az` CLI is a wrapper. Terraform is a wrapper. The portal is a wrapper. They all hit the same REST API.

Useful side-effect: every action has an **Activity Log** entry. Need to know who deleted the cluster? Check Activity Log.

9.8 Gotchas (a partial list)

- **Region capacity.** Some VM SKUs are unavailable in your region/subscription. `az vm list-skus -l eastus2 --output table` to check.
- **Quota.** New subscriptions have low CPU quotas. You'll get "QuotaExceeded" and need to file a support ticket.
- **Delete protection.** RGs and locks can be set to "CanNotDelete." Terraform `destroy` will fail until removed.
- **Public IP SKU mismatch.** Basic LB needs Basic PIP. Standard LB needs Standard PIP. Mix them and you get a cryptic error.
- **Subscription ID hardcoding.** Bad — different envs different subs. Use `data "azurerm_subscription" "current"` instead.

- **AAD vs AAD B2C vs AAD External ID.** Three different things. For platform-engineering interviews, “Microsoft Entra ID” (formerly AAD) is the one.

9.9 In our lab

- **Tenant + subscription** — your personal subscription.
- **Resource groups** — `gskplat-prod-rg`, `gskplat-dev-rg`, plus `gskplat-bootstrap-rg` for state storage.
- **SP** — `sp-terraform` with Contributor + User Access Administrator on the subscription.
- **AKS** — `gskplataksn73d5y` (kubenet, B2s nodes, 1-3 node autoscale).
- **ACR** — `gskplatacrn73d5y` (attached to AKS, allows `AcrPull` from the cluster’s MI).
- **Log Analytics Workspace** — `gskplat-law` for container logs and Azure Monitor.

9.10 Interview talking points

Q: “How does AKS auth to ACR?”

“When you create AKS with `--attach-acr`, Azure grants the AKS cluster’s kubelet identity the `AcrPull` role on the registry. After that, image pulls just work — no `imagePullSecrets` needed. Behind the scenes it’s a role assignment at the ACR scope, granted to the kubelet’s managed identity.”

Q: “Service Principal vs Managed Identity?”

“SP is for workloads outside Azure — a laptop, GitHub Actions, a CI server somewhere. Has a client ID + secret you have to rotate. MI is for workloads running in Azure — VM, AKS, Function. Azure manages the credential rotation for you. **Always prefer MI when the workload is in Azure.**”

Q: “What’s the difference between an RG and a subscription?”

“Subscription is the billing boundary — everything in it bills to the same Azure agreement. RG is a logical grouping inside one subscription — deleting an RG deletes everything in it. Roles can be assigned at either scope. For multi-team setups: one subscription per team or per environment is common.”

9.11 Exercises

1. `az account show` — find your subscription ID.
2. Create an RG with `az group create`. Then with Terraform. Compare.
3. Create an SP with `az ad sp create-for-rbac`. Use it to log in. Delete it.
4. Create an AKS cluster manually. Then with Terraform. Compare time and YAML.
5. Push an image to ACR with `az acr build`. Pull it from a different machine.
6. Make a deliberate NSG mistake (block 443 inbound). Observe what breaks.
7. Set a `CanNotDelete` lock on an RG. Try `terraform destroy`. Remove the lock.

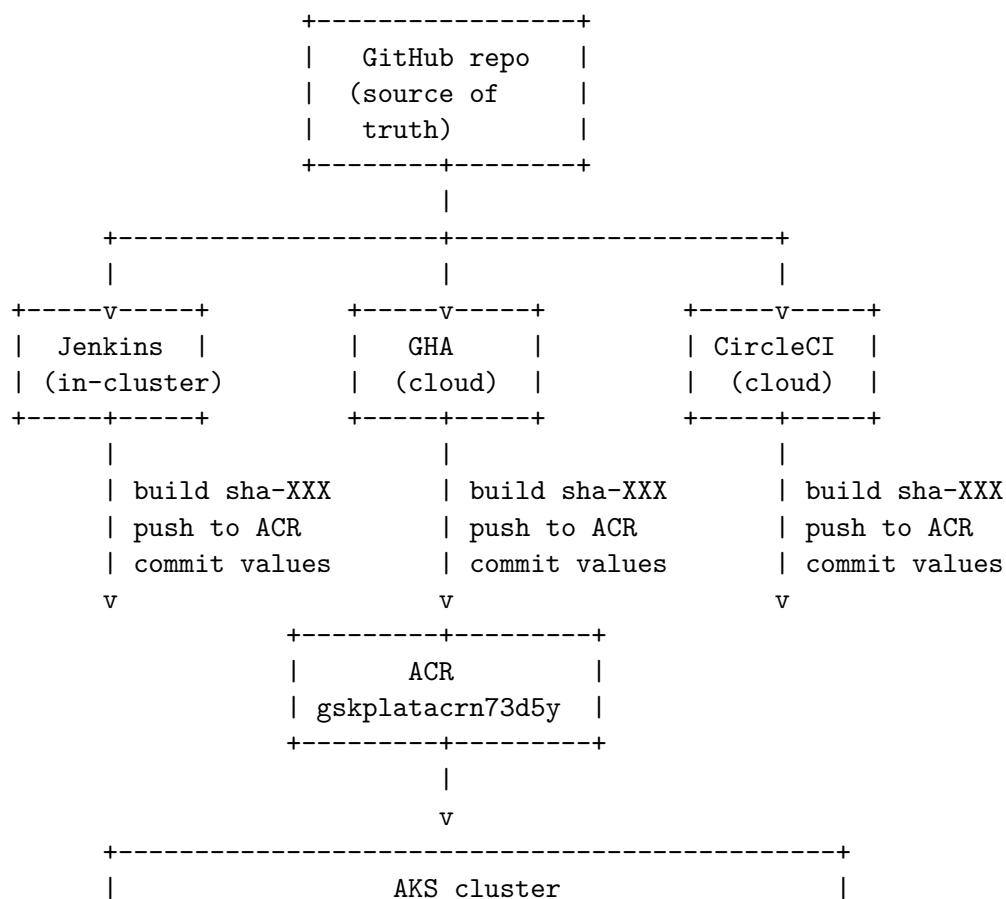
Part II — The Lab, End to End

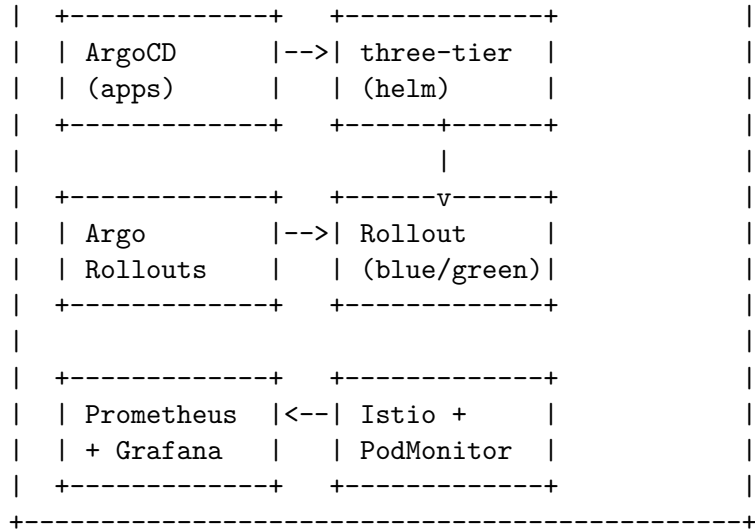
Chapter 10: Lab Architecture Overview

10.1 The 30-second pitch

The lab is a working production-shaped platform you can stand up on Azure with `terraform apply` and `kubectl apply`. Three CI systems push to one cluster. ArgoCD reconciles. Prometheus watches. Argo Rollouts gates promotions. It's small enough to run on a B2s, big enough to talk about in an interview.

10.2 The big picture





10.3 Components inventory

Layer	Component	What it does
Infra (Terraform)	Resource Groups, VNet, NSGs, AKS, ACR, Log Analytics	The Azure substrate
Platform (Helm/manifests)	Istio, ArgoCD, Argo Rollouts, kube-prometheus-stack, Jenkins	The shared services
App (Helm)	three-tier umbrella: frontend (nginx), backend (httpd), database (postgres)	What we deploy
CI	Jenkins (in-cluster), GitHub Actions, CircleCI	Three independent CI demonstrations
GitOps	ArgoCD apps + Application sets	Watches Git, reconciles cluster
Progressive delivery	Argo Rollouts + AnalysisTemplate	Blue/green with Prometheus gate
Observability	Prometheus, Grafana, PodMonitor for Istio	Metrics + dashboards

10.4 The repository layout

```

azure-platform-lab/
+-- infra/                # Terraform
|   +-- bootstrap/        # state storage (run once)
|   +-- modules/
|       +-- network/      # VNet, subnets, NSGs
|       +-- aks/
|       +-- acr/
|       +-- monitoring/

```

```

|   +-- live/
|       +-- dev/
|       +-- prod/
+-- kubernetes/
|   +-- platform/                # platform component manifests
|       |   +-- argocd/
|       |   +-- argo-rollouts/
|       |   +-- istio/
|       |   +-- prometheus/
|   +-- apps/
|       +-- three-tier/
|           +-- Chart.yaml
|           +-- values.yaml
|           +-- values-dev.yaml
|           +-- values-prod.yaml
|           +-- charts/
|               +-- frontend/
|               +-- backend/
|               +-- database/
+-- ci/
|   +-- jenkins/Jenkinsfile
|   +-- .github/workflows/ci-cd.yaml
|   +-- .circleci/config.yml
+-- docs/
|   +-- textbook/
|       +-- platform-engineering-handbook.md
|       +-- glossary-and-tips.md
|       +-- Makefile
+-- README.md

```

10.5 What you can demo

- **End-to-end build:** push a commit, watch CI build, see ArgoCD apply, see rollout progress.
- **Blue/green gate:** trigger a deploy with a bad backend, watch the analysis fail and rollout pause.
- **Three CIs, one cluster:** the same change ships via Jenkins, GHA, or CircleCI — all converge on the same Git state.
- **Drift remediation:** edit a Deployment directly in the cluster, watch ArgoCD revert it (selfHeal).
- **Observability:** Grafana dashboards for Istio mesh, rollout success rate, request latency.

10.6 Costs

A rough monthly idle cost in eastus2:

Resource	~USD/month
AKS control plane	free (uptime tier)
1x Standard_B2s node	~\$30
1x Public IP (Standard)	~\$4
ACR Basic	~\$5
Log Analytics (low ingest)	~\$5
Storage (state + logs)	~\$1
Total	~\$45

Tear it down when you're not using it: `terraform destroy` everything except `bootstrap/` (state storage). Stand back up in ~15 minutes.

10.7 What this lab is *not*

- Not multi-region or HA.
- Not network-isolated (public LB, no private endpoints).
- Not paying for premium ACR features (geo-replication, content trust).
- Not running on managed Postgres (we use a Postgres StatefulSet for cost).

These are the production gaps we discuss in [Chapter 20](#).

Chapter 11: Iteration 1 — Bootstrap and Network

11.1 Goal of this iteration

Get the foundation in place: a remote state backend, a resource group, networking, and observability plumbing. **No cluster yet.** This iteration is about giving every later iteration a place to stand.

11.2 What we create

```
+-----+
| Bootstrap RG          |
| +-- storage account    |    <-- holds tfstate blobs
| +-- container "tfstate" |
+-----+

+-----+
| Workload RG (gskplat-rg) |
| +-- VNet (10.42.0.0/16)  |
| |   +-- subnet aks (10.42.1.0/24)
| |   +-- subnet svc (10.42.2.0/24)
| +-- NSG (attached to aks subnet)
| +-- Log Analytics workspace
+-----+
```

11.3 Why this order

You can't store Terraform state in a backend that doesn't exist yet. Chicken-and-egg. The fix is **bootstrap**:

Step 1: `bootstrap/` uses `LOCAL` state to create the storage account.

Step 2: We commit the bootstrap state file (it has no secrets).

Step 3: `live/` uses `REMOTE` state pointing at that storage account.

After bootstrap, every other Terraform configuration uses remote state. You never touch the bootstrap state file again — it sits there as a tiny config.

11.4 The bootstrap

```
# infra/bootstrap/main.tf
resource "azurerm_resource_group" "tfstate" {
  name      = "gskplat-bootstrap-rg"
  location  = var.location
}

resource "azurerm_storage_account" "tfstate" {
  name                        = "gskplattfstate" # globally unique
  resource_group_name        = azurerm_resource_group.tfstate.name
  location                   = var.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"

  blob_properties {
    versioning_enabled = true # state has secrets, keep history
  }
}

resource "azurerm_storage_container" "tfstate" {
  name                  = "tfstate"
  storage_account_name = azurerm_storage_account.tfstate.name
  container_access_type = "private"
}
```

11.5 The live config

```
# infra/live/prod/backend.tf
terraform {
  backend "azurerm" {
    resource_group_name = "gskplat-bootstrap-rg"
    storage_account_name = "gskplattfstate"
    container_name       = "tfstate"
    key                  = "prod.tfstate"
  }
}

# infra/live/prod/main.tf
module "network" {
  source      = "../../modules/network"
  resource_group_name = azurerm_resource_group.workload.name
  location    = var.location
  address_space = ["10.42.0.0/16"]
  subnets = {
    aks = "10.42.1.0/24"
    svc = "10.42.2.0/24"
  }
}
```

```

}

module "monitoring" {
  source           = "../../modules/monitoring"
  resource_group_name = azurerm_resource_group.workload.name
  location         = var.location
  workspace_name    = "gskplat-law"
}

```

11.6 NSG rules — get them right early

Our NSG starts with the implicit rules: AllowVnetInBound, AllowAzureLoadBalancerInBound, DenyAllInBound. We **don't** add an explicit DenyAll. We only add an allow for SSH from our home IP if we plan to put a jumpbox in the subnet (we don't for AKS subnets).

See gotcha in 9.3. We learned this the hard way.

11.7 Verifying

```

cd infra/bootstrap && terraform init && terraform apply
cd ../live/prod && terraform init && terraform plan
terraform apply

```

After this, you should have:

- A storage account with a `tfstate` container
- A workload RG with an empty VNet
- A Log Analytics workspace ready to receive logs

11.8 What iteration 1 *doesn't* have

- No cluster.
- No registry.
- No apps.

It's just the substrate. Move to iteration 2 to get a cluster.

Chapter 12: Iteration 2 — ACR and AKS

12.1 Goal

Stand up the cluster and the registry, attach them, and verify you can deploy a “hello world” pod.

12.2 What we create

```
+-----+
| RG: gskplat-rg          |
|   +--- ACR (gskplatacrn73d5y) |
|   +--- AKS (gskplataksn73d5y) |
|       |                   |
|       +--- system node pool (1x Standard_B2s)
|       +--- (later) user node pool
|       +--- kubelet MI gets AcrPull on ACR
+-----+
```

12.3 The AKS module

Key choices we lock in:

Choice	Value	Why
Region	eastus2	Cheap, ample SKU availability
Node SKU	Standard_B2s	2 vCPU, 4 GB — fits in free credit
Network	kubenet	Lower IP usage, fine for one cluster
Identity	System-assigned MI	One less thing to manage
RBAC	Azure RBAC	Use AAD identities for kubectl

```
# infra/modules/aks/main.tf
resource "azurerm_kubernetes_cluster" "this" {
  name                = var.name
  location             = var.location
  resource_group_name = var.resource_group_name
  dns_prefix          = var.name
```

```

kubernetes_version = var.kubernetes_version

default_node_pool {
  name           = "system"
  node_count     = 1
  vm_size        = "Standard_B2s"
  vnet_subnet_id = var.aks_subnet_id
}

identity {
  type = "SystemAssigned"
}

network_profile {
  network_plugin = "kubenet"
}

oms_agent {
  log_analytics_workspace_id = var.law_id
}
}

```

12.4 Attaching ACR to AKS

This was the line that taught us about User Access Administrator:

```

resource "azurerm_role_assignment" "aks_acr_pull" {
  scope                = azurerm_container_registry.this.id
  role_definition_name = "AcrPull"
  principal_id         = azurerm_kubernetes_cluster.this.kubelet_identity[0].object_id
}

```

For this `role_assignment` to succeed, the Terraform SP needs `Microsoft.Authorization/roleAssignments/write` — which **Contributor doesn't have**. Add **User Access Administrator** on the subscription, and it works.

12.5 Getting credentials

```

az aks get-credentials -g gskplat-rg -n gskplataksn73d5y
kubectl get nodes

```

#	NAME	STATUS	ROLES	AGE	VERSION
#	aks-system-12345678-vmss000000	Ready	agent	2m	v1.29.x

12.6 Smoke test

```

kubectl run hello --image=gskplatacrn73d5y.azurecr.io/library/nginx:alpine --restart=Never
kubectl get pod hello

```

```
kubectl logs hello
kubectl delete pod hello
```

If the image pull works, ACR-to-AKS auth is wired. If it doesn't:

Symptom	Likely cause
ImagePullBackOff + “unauthorized”	AcrPull role not yet propagated (wait 2 min) or wrong principal
ErrImagePull + “not found”	Image not in ACR (push it first with <code>az acr import</code>)
Pod never schedules	Node pool is cordoned , NSG mistake, or out of capacity

12.7 What iteration 2 *doesn't* have

- No Ingress (we expose later via Istio).
- No platform tooling (ArgoCD, Prometheus come in iteration 3).
- No real app (iteration 4).

You can already deploy raw YAML to it. The next iteration makes the cluster manageable.

Chapter 13: Iteration 3 — Platform Components

13.1 Goal

Install the shared platform services: Istio (mesh + ingress), ArgoCD (GitOps), Argo Rollouts (progressive delivery), Prometheus + Grafana (observability), and Jenkins (in-cluster CI for the demo).

13.2 The order matters

1. Istio (provides ingress for the rest)
2. ArgoCD (manages itself + everything else once installed)
3. Argo Rollouts (needed before any Rollout-based chart)
4. kube-prometheus-stack (CRDs first; PodMonitors come later)
5. Jenkins (uses Istio gateway, scrapes via Prometheus)

We install each via Helm or kubectl-applied manifests. After ArgoCD is up, ArgoCD applies the rest (manage itself, then everyone else).

13.3 Istio (ingress only mode)

We use Istio as the cluster ingress. mTLS is set to PERMISSIVE so non-mesh pods can still call each other while we adopt incrementally.

```
# kubernetes/platform/istio/istio-operator.yaml (excerpt)
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: default
  meshConfig:
    enableAutoMtls: true
  components:
    ingressGateways:
      - name: istio-ingressgateway
        enabled: true
        k8s:
```

```

        service:
          type: LoadBalancer

# kubernetes/platform/istio/gateway.yaml
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: main-gateway
  namespace: istio-system
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"

```

The LoadBalancer Service gets a public IP from Azure. That's the cluster's front door.

13.4 ArgoCD

```
kubectl create ns argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests
```

Then expose via VirtualService and create our app-of-apps:

```

# kubernetes/platform/argocd/root-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/gopalskhandale/azure-platform-lab
    path: kubernetes/platform
    targetRevision: HEAD
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
      prune: false           # we explicitly opt-in to prune later

```

After this, ArgoCD watches the `kubernetes/platform/` directory and reconciles everything underneath. The pattern is called **App of Apps**: the root app deploys other Application objects.

13.5 Argo Rollouts

```
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts \
  -f https://github.com/argoproj/argo-rollouts/releases/latest/download/install.yaml
```

Argo Rollouts gives us a Rollout CRD that behaves like a Deployment but adds **blue/green** and **canary** strategies plus **AnalysisTemplate** for promotion gating. Iteration 6 wires it into the three-tier app.

13.6 kube-prometheus-stack

Helm:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  -f values.yaml
```

The chart installs Prometheus Operator, Prometheus, Alertmanager, Grafana, and all the CRDs (ServiceMonitor, PodMonitor, PrometheusRule).

A PodMonitor is what we use for Istio — Envoy sidecars expose `/stats/prometheus` on port 15090:

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: envoy-stats-monitor
  namespace: istio-system
spec:
  selector:
    matchExpressions:
      - { key: istio-prometheus-ignore, operator: DoesNotExist }
  namespaceSelector:
    any: true
  jobLabel: envoy-stats
  podMetricsEndpoints:
    - path: /stats/prometheus
      interval: 15s
      relabelings:
        - action: keep
          sourceLabels: [__meta_kubernetes_pod_container_name]
          regex: istio-proxy
```

13.7 Jenkins (in-cluster)

We deploy Jenkins via the official Helm chart with kaniko agents. The interesting bits:

- ServiceAccount with `system:auth-delegator` (cluster-scoped).
- A volume mount for the ACR docker config (so kaniko can push).

- An in-cluster `jenkins` Service, fronted by an Istio `VirtualService`.

Why in-cluster? Because then Jenkins can run `kubectl/helm` directly against the cluster API without external auth, demonstrating a “CI lives next to the workload” pattern.

13.8 Verifying

After iteration 3, `kubectl get apps -n argocd` shows several Applications, all healthy and synced:

NAME	SYNC STATUS	HEALTH
root	Synced	Healthy
istio	Synced	Healthy
argo-rollouts	Synced	Healthy
kube-prometheus	Synced	Healthy
jenkins	Synced	Healthy

Open Grafana via port-forward → see the kube-state-metrics dashboards. Open ArgoCD UI → see the app tree. Open Jenkins → log in with the bootstrap admin password from a Secret.

13.9 What iteration 3 *doesn't* have yet

- No application code.
- No CI pipelines wired to a real app.
- No progressive delivery in practice (Rollouts is installed but not used).

Iteration 4 adds the app.

Chapter 14: Iteration 4 — Three-Tier App (Helm)

14.1 Goal

Define the application — frontend, backend, database — as a Helm umbrella chart, with separate `values-dev.yaml` and `values-prod.yaml` for environment differences.

14.2 The shape

```
+-----+ HTTP +-----+ TCP +-----+
| frontend |----->| backend |----->| database |
| (nginx)  |         | (httpd)  |         | (postgres)
| Rollout  |         | Rollout  |         | StatefulSet
+-----+         +-----+         +-----+
```

```
  ^
  |
  | Ingress (Istio Gateway + VirtualService)
  |
public IP
```

- **frontend** — nginx, serves a tiny HTML page with a “greeting” injected from values. Calls the backend on `/api`.
- **backend** — httpd, echos a message + headers. Has a `/health` endpoint.
- **database** — postgres 15-alpine StatefulSet with a PVC.

14.3 Chart layout

```
kubernetes/apps/three-tier/
Chart.yaml
values.yaml          <-- defaults
values-dev.yaml      <-- dev overrides
values-prod.yaml     <-- prod overrides
charts/
  frontend/
    Chart.yaml
    values.yaml
    templates/
```

```

    rollout.yaml
    service.yaml
    configmap.yaml
    virtualservice.yaml
backend/
  Chart.yaml
  values.yaml
  templates/
    rollout.yaml
    service.yaml
database/
  Chart.yaml
  values.yaml
  templates/
    statefulset.yaml
    service.yaml
    secret.yaml

```

The umbrella Chart.yaml declares dependencies on the three subcharts.

14.4 Values pattern

```

# values.yaml (defaults)
frontend:
  image:
    repository: nginx
    tag: latest
  replicaCount: 1
  greeting: Hello from Helm
  resources:
    requests: { cpu: 30m, memory: 32Mi }
    limits:   { cpu: 100m, memory: 128Mi }
backend:
  image:
    repository: httpd
    tag: alpine
  replicaCount: 1
database:
  image:
    repository: postgres
    tag: 15-alpine
  database: appdb
  username: appuser
  password: change-me

# values-prod.yaml (overrides)
frontend:
  image:

```

```

    repository: gskplatacrn73d5y.azurecr.io/three-tier/frontend
    tag: sha-9b61859
  greeting: Hello from PROD
backend:
  image:
    repository: gskplatacrn73d5y.azurecr.io/three-tier/backend
    tag: sha-9b61859
  env:
    APP_ENV: prod

```

14.5 Frontend template (without rollouts yet)

```

apiVersion: apps/v1
kind: Deployment      # <-- becomes Rollout in iter 6
metadata:
  name: {{ .Release.Name }}-frontend
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          resources: {{ toYaml .Values.resources | nindent 10 }}
          ports:
            - containerPort: 80

```

14.6 Database secret pattern

The Postgres password is rendered from values into a Secret. **Real prod would use external-secrets**, but for the lab the password is in values-*.yaml (and .gitignore covers the textbook example).

```

apiVersion: v1
kind: Secret
metadata:
  name: {{ .Release.Name }}-db
type: Opaque
stringData:
  POSTGRES_USER: {{ .Values.username }}
  POSTGRES_PASSWORD: {{ .Values.password }}
  POSTGRES_DB: {{ .Values.database }}

```

14.7 Installing via ArgoCD

The ArgoCD Application points at this chart and a values file:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: three-tier-prod
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/gopalskhandale/azure-platform-lab
    path: kubernetes/apps/three-tier
    helm:
      valueFiles:
        - values-prod.yaml
  destination:
    namespace: app-prod
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      selfHeal: true
```

`kubectl get app -n argocd three-tier-prod` shows it Synced + Healthy when everything lines up.

14.8 Verifying end-to-end

```
# get the ingress IP
kubectl get svc -n istio-system istio-ingressgateway

# curl through it
curl -H "Host: app.example.com" http://<EXTERNAL-IP>/
# -> "Hello from PROD"

curl -H "Host: app.example.com" http://<EXTERNAL-IP>/api/
# -> backend response from prod
```

14.9 What iteration 4 *doesn't* have

- No CI yet — image tags are set manually.
- No blue/green — straight rolling updates.
- No analysis gate — bad deploys go live immediately.

That all lands in iterations 5 and 6.

Chapter 15: Iteration 5 — Real Code + Jenkins Pipeline

15.1 Goal

Wire a real CI pipeline. Build the frontend and backend images with **kaniko** (in-cluster, no Docker socket), push to ACR, run tests, scan with Trivy, update Helm values, and let ArgoCD apply.

15.2 Why kaniko

The cluster doesn't have Docker available to pods, and we don't want to mount the host Docker socket (security hole). **Kaniko builds container images inside a container** using userspace tooling — no daemon needed.

```
+-----+ +-----+ +-----+
| Jenkins pod |--> | kaniko  |--> | ACR  |
| (orchestr.) |   | builds  |   |      |
+-----+ | + pushes| +-----+
           +-----+
```

15.3 The Jenkinsfile shape

```
pipeline {
  agent {
    kubernetes {
      yml '''
apiVersion: v1
kind: Pod
spec:
  serviceAccountName: jenkins
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:latest
  - name: kaniko-backend
    image: gcr.io/kaniko-project/executor:latest
    command: [sleep]
    args: [99999]
```

```

    volumeMounts:
    - name: docker-config
      mountPath: /kaniko/.docker
  - name: kaniko-frontend
    image: gcr.io/kaniko-project/executor:latest
    command: [sleep]
    args: [99999]
    volumeMounts:
    - name: docker-config
      mountPath: /kaniko/.docker
  - name: tools
    image: alpine/k8s:1.29.0
    command: [sleep]
    args: [99999]
volumes:
- name: docker-config
  secret:
    secretName: acr-docker-config
    items:
    - key: .dockerconfigjson
      path: config.json
'''
}
}
environment {
  SHA = sh(returnStdout: true, script: "git rev-parse --short=7 HEAD").trim()
}
stages {
  stage('Build images') {
    parallel {
      stage('backend') {
        steps {
          container('kaniko-backend') {
            sh '''
              /kaniko/executor \
                --context apps/backend \
                --destination $ACR/three-tier/backend:sha-$SHA \
                --cache=true --cache-repo=$ACR/cache/backend
            '''
          }
        }
      }
      stage('frontend') {
        steps {
          container('kaniko-frontend') {
            sh '''
              /kaniko/executor \
                --context apps/frontend \

```

```

        --destination $ACR/three-tier/frontend:sha-$SHA \
        --cache=true --cache-repo=$ACR/cache/frontend
    ''
    }
  }
}
}
stage('Scan') {
  steps {
    container('tools') {
      sh 'trivy image --exit-code 0 $ACR/three-tier/backend:sha-$SHA'
    }
  }
}
stage('Update Helm values') {
  steps {
    container('tools') {
      sh '''
        sed -i "s|tag: .*|tag: sha-$SHA|" \
        kubernetes/apps/three-tier/values-dev.yaml
        git commit -am "ci(dev): bump images to sha-$SHA"
        git push origin master
      '''
    }
  }
}
}
}
}
}
}

```

15.4 Gotchas we hit

- `timestamps()` required a plugin we didn't have. Removed it.
- `kaniko --cleanup` caused the container to exit after build, breaking the parallel stages. Removed it.
- **Single kaniko container** with two builds shared state badly. **Split into kaniko-backend and kaniko-frontend.**
- **ACR Docker config** had to be a Secret with `.dockerconfigjson`, mounted into `/kaniko/.docker/config.json`. Kaniko expects exactly that path.

15.5 Why a Git SHA tag and not latest

Tag	Immutable?	Promotable?	Reproducible?
latest	no	no	no
v1.0.0	usually	yes	yes
sha-9b61859	yes	yes	yes

Git SHA is the perfect immutable image tag. It also lets us **promote the same image** from dev to prod by reusing the SHA — no rebuild between environments.

15.6 Manual trigger or every commit?

We chose **manual trigger** (`workflow_dispatch` in GHA, “Build Now” in Jenkins). Reasons:

- Lab usage. We don’t want every README typo to spin up kaniko.
- Easier to demo. Click button, watch pipeline.
- Production would obviously be on push, with PR gates.

15.7 The end-to-end flow

1. dev pushes to master
2. dev clicks "Build" in Jenkins
3. Jenkins pipeline:
 - a. computes SHA
 - b. builds backend + frontend with kaniko in parallel
 - c. pushes to ACR
 - d. scans with Trivy (report only)
 - e. edits `values-dev.yaml`: `tag -> sha-XXX`
 - f. commits + pushes to master
4. ArgoCD notices the commit (within 3 min auto-poll, or webhook)
5. ArgoCD re-renders helm, applies to cluster
6. Pods restart with new image

15.8 What we still don’t have

- Blue/green strategy.
- A “promote dev to prod” step.
- A second CI to prove this is portable.

Iteration 6 brings the first two. Iteration 7 brings the third.

Chapter 16: Iteration 6 — GitHub Actions + Argo Rollouts + Prometheus Gate

16.1 Goal

Three things in this iteration:

1. **A second CI** (GitHub Actions) to prove the build-and-commit pattern is portable.
2. **Argo Rollouts blue/green** strategy on the frontend and backend.
3. **A Prometheus-based pre-promotion gate** that auto-fails bad deploys.

16.2 GitHub Actions pipeline

```
# .github/workflows/ci-cd.yaml (skeleton)
name: ci-cd
on:
  workflow_dispatch:      # manual trigger

permissions:
  contents: write          # so GITHUB_TOKEN can push values changes

jobs:
  resolve-tag:
    runs-on: ubuntu-latest
    outputs:
      sha: ${ steps.s.outputs.sha }
    steps:
      - uses: actions/checkout@v4
      - id: s
        run: echo "sha=sha-$(git rev-parse --short=7 HEAD)" >> $GITHUB_OUTPUT

  build-backend:
```

```

- uses: actions/checkout@v4
- uses: docker/setup-buildx-action@v3
  with:
    driver-opts: image=moby/buildkit:latest
- run: docker buildx inspect --bootstrap # containerd-snapshotter
- uses: azure/docker-login@v1
  with:
    login-server: ${ secrets.ACR }
    username: ${ secrets.ACR_USER }
    password: ${ secrets.ACR_PASS }
- uses: docker/build-push-action@v5
  with:
    context: apps/backend
    push: true
    tags: ${ secrets.ACR }/three-tier/backend:${ needs.resolve-tag.outputs.sha }
    cache-from: type=gha,scope=backend
    cache-to: type=gha,scope=backend,mode=max

update-dev-tags:
  needs: [build-backend, build-frontend]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        token: ${ secrets.GITHUB_TOKEN }
    - run: |
        pip install pyyaml
        python - <<'PY'
        import yaml, pathlib
        tag = "${ needs.resolve-tag.outputs.sha }"
        f = pathlib.Path("kubernetes/apps/three-tier/values-dev.yaml")
        y = yaml.safe_load(f.read_text())
        y["frontend"]["image"]["tag"] = tag
        y["backend"]["image"]["tag"] = tag
        f.write_text(yaml.safe_dump(y))
        PY
    - run: |
        git config user.name "github-actions"
        git config user.email "actions@github.com"
        git commit -am "ci(dev): bump images to ${ needs.resolve-tag.outputs.sha } [skip ci]"
        git push

approve-prod:
  needs: update-dev-tags
  runs-on: ubuntu-latest
  environment: prod-approval # GitHub Environments adds a manual gate
  steps:
    - run: echo "approved"

```

```

update-prod-tags:
  needs: approve-prod
  runs-on: ubuntu-latest
  # ... same as update-dev-tags but on values-prod.yaml ...

```

Things we hit and fixed

- **Buildx “Cache export is not supported for the docker driver.”** Fix: enable the containerd image store on the runner (containerd-snapshotter) so the standard docker driver supports caching.
- **PAT vs GITHUB_TOKEN.** Started with a PAT, switched to GITHUB_TOKEN + permissions: contents: write. Simpler, no rotation.
- **Trivy CVEs.** alpine images have known CVEs that can’t be patched without breaking the image. We moved Trivy to **report-only** (exit-code: "0"). Documented as a known gap.

16.3 Argo Rollouts blue/green

The frontend Rollout (formerly Deployment):

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: {{ .Release.Name }}-frontend
spec:
  replicas: {{ .Values.replicaCount }}
  strategy:
    blueGreen:
      activeService: {{ .Release.Name }}-frontend-active
      previewService: {{ .Release.Name }}-frontend-preview
      autoPromotionEnabled: {{ .Values.blueGreen.autoPromotionEnabled }}
      scaleDownDelaySeconds: 30
      prePromotionAnalysis:
        templates:
          - templateName: frontend-success-rate
  selector:
    matchLabels: { app: frontend }
  template: { ... }

```

What “blue/green” means here

Before deploy:

```

+-----+ active -> v1 (blue)
| user |-----+
+-----+
                                     |
                                     v
                               +-----+
                               | v1 pods (4) |

```

```

+-----+
Mid-deploy (analysis running):
+-----+ active -> v1 (blue)
| user |-----+
+-----+
                                     v
                                     +-----+
                                     | v1 pods (4) | <-- still serving traffic
                                     +-----+
                                     +-----+
                                     | v2 pods (4) | <-- new, being analyzed
                                     +-----+
                                     ^
                                     | preview svc routes here for analysis

```

If analysis passes:

- active svc switches to v2
- v1 stays up for `scaleDownDelaySeconds` (30s) for fast rollback
- then v1 scales down

If analysis fails:

- active svc never switches
- v2 pods scale down
- rollout marked Degraded

16.4 AnalysisTemplate — the gate

```

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: frontend-success-rate
spec:
  metrics:
  - name: min-traffic
    successCondition: result >= 1
    provider:
      prometheus:
        address: http://kps-prometheus.monitoring.svc:9090
        query: |
          sum(rate(istio_requests_total{
            destination_workload="three-tier-prod-frontend",
            reporter="destination"
          }[1m])) or vector(0)
  - name: success-rate
    successCondition: result >= 0.95
    provider:
      prometheus:

```

```

address: http://kps-prometheus.monitoring.svc:9090
query: |
    sum(rate(istio_requests_total{
        destination_workload="three-tier-prod-frontend",
        response_code!~"5..",
        reporter="destination"
    }[1m]))
    /
    sum(rate(istio_requests_total{
        destination_workload="three-tier-prod-frontend",
        reporter="destination"
    }[1m])) or vector(1)

```

Why two metrics

- **success-rate alone** passes trivially at zero traffic. Bad deploy + no traffic = false green.
- **min-traffic gate** requires 1 req/sec before we trust the success rate.
- **or vector(1) / or vector(0)** keeps Prometheus from returning an empty vector (which crashes the analysis controller with “slice index out of range”).

Both metrics must pass for the rollout to promote.

16.5 Auto-promote dev, manual-promote prod

```

# values-dev.yaml
blueGreen:
  autoPromotionEnabled: true    # fast feedback in dev

# values-prod.yaml
blueGreen:
  autoPromotionEnabled: false   # human eyes before prod cutover

```

In prod, after the analysis passes, the Rollout sits in “Paused” state. An operator runs:

```
kubectl argo rollouts promote three-tier-prod-frontend -n app-prod
```

...or clicks “Promote” in the Argo Rollouts dashboard.

16.6 Triggering a failure on purpose (to verify the gate)

1. Push a backend image that 500s on every request.
2. CI bumps values-prod.yaml to the new tag.
3. ArgoCD applies.
4. Rollout starts. v2 pods come up.
5. Analysis runs at 30s intervals. min-traffic is 0 → fails first run.
6. After 3 failed checks, Rollout is **Degraded**. Active service never switches.
7. v2 pods stay around (for rollback) but get no real traffic.

This is the most satisfying part of the lab to demo.

16.7 What iteration 6 *doesn't* have

- Only two CIs. The portfolio story is “here’s the GitOps pattern across N tools.” We add CircleCI in iteration 7.
-

Chapter 17: Iteration 7 — CircleCI

17.1 Goal

A third CI system, on a cloud-hosted runner, with manual approval gates. Demonstrates that the GitOps pattern — build, commit values, let ArgoCD apply — is CI-tool-agnostic.

17.2 CircleCI vocabulary in 60 seconds

CircleCI term	What it is
Orb	A reusable, packaged set of commands/jobs (like Helm chart for CI)
Context	A named bag of environment variables/secrets, scoped at the org
Workflow	An ordered/branching set of jobs
Approval job	type: approval — pauses the workflow until a human clicks
Workspace	Files/data passed between jobs via persist_to_workspace / attach_workspace
Executor	Where the job runs: docker / machine / macos

17.3 The config

```
version: 2.1

orbs:
  node: circleci/node@5.2.0

executors:
  default:
    docker:
      - image: cimg/base:current

jobs:
  resolve-tag:
    executor: default
    steps:
```

```

- checkout
- run:
  name: Compute SHA tag
  command: |
    SHA=$(git rev-parse --short=7 HEAD)
    mkdir -p workspace
    printf "sha-%s" "$SHA" > workspace/image_tag
    cat workspace/image_tag
- persist_to_workspace:
  root: workspace
  paths: [image_tag]

build-and-push:
  executor: default
  parameters:
    service:
      type: string
  steps:
    - checkout
    - setup_remote_docker
    - attach_workspace: { at: workspace }
    - run:
      name: Login to ACR
      command: |
        echo "$ACR_PASSWORD" | docker login "$ACR_LOGIN_SERVER" \
          -u "$ACR_USERNAME" --password-stdin
    - run:
      name: Build and push <<parameters.service>>
      command: |
        TAG=$(cat workspace/image_tag)
        docker build -t "$ACR_LOGIN_SERVER/three-tier/<<parameters.service>>:$TAG" \
          apps/<<parameters.service>>
        docker push "$ACR_LOGIN_SERVER/three-tier/<<parameters.service>>:$TAG"

bump-values:
  executor: default
  parameters:
    env_file:
      type: string
  steps:
    - checkout
    - attach_workspace: { at: workspace }
    - run:
      name: Update values
      command: |
        pip install --quiet pyyaml
        TAG=$(cat workspace/image_tag)
        python3 -c "

```

```

import yaml, pathlib, os
tag = os.environ['TAG']
f = pathlib.Path('kubernetes/apps/three-tier/.<<parameters.env_file>>')
y = yaml.safe_load(f.read_text())
y['frontend']['image']['tag'] = tag
y['backend']['image']['tag'] = tag
f.write_text(yaml.safe_dump(y))
"

git config user.name "circleci"
git config user.email "ci@circleci.local"
git commit -am "ci(circleci): bump <<parameters.env_file>> to $TAG [skip ci]"
git push https://$GITHUB_TOKEN@github.com/gopalskhandale/azure-platform-lab master

```

workflows:

```

build-test-deploy:
  jobs:
    - resolve-tag
    - build-and-push:
        name: build-backend
        service: backend
        context: azure-lab
        requires: [resolve-tag]
    - build-and-push:
        name: build-frontend
        service: frontend
        context: azure-lab
        requires: [resolve-tag]
    - bump-values:
        name: bump-dev
        env_file: values-dev.yaml
        context: github-write
        requires: [build-backend, build-frontend]
    - hold-for-prod:
        type: approval
        requires: [bump-dev]
    - bump-values:
        name: bump-prod
        env_file: values-prod.yaml
        context: github-write
        requires: [hold-for-prod]

```

17.4 Contexts (where secrets live)

We define two Contexts in the CircleCI org settings:

Context	Variables
azure-lab	ACR_LOGIN_SERVER, ACR_USERNAME, ACR_PASSWORD
github-write	GITHUB_TOKEN (a PAT with repo:write)

Each job lists the contexts it needs. Secrets are scoped: the build job has ACR creds but no git write; the bump job has git write but no ACR creds.

17.5 The approval gate

`hold-for-prod` is a `type: approval` job — no executor, no steps. The workflow pauses there. In the CircleCI UI you click “Approve” or “Cancel.” Approve → `bump-prod` runs.

This mirrors GHA’s `environment: prod-approval` and Jenkins’ `input` step. Three different syntaxes, same concept.

17.6 Gotchas we hit

- **<<EOF heredoc.** CircleCI’s YAML uses `<<` for parameter syntax — your shell heredoc collides. Replace with `printf` or `python3 -c "..."` strings.
- **ModuleNotFoundError: yaml.** The `cimg/base` image doesn’t ship `pyyaml`. `pip install --quiet pyyaml` in the step.
- **Image tag as an output.** No outputs in CircleCI jobs. Use `persist_to_workspace / attach_workspace` instead.

17.7 What you can demo

- Manual trigger of the workflow from the CircleCI UI.
- Approval gate before prod.
- Same Git SHA flows from dev (auto) to prod (after approval).
- The dev/prod bumps commit to the same repo ArgoCD watches.
- Within minutes of the prod bump, ArgoCD reconciles and the Rollout’s analysis runs.

17.8 Cross-CI table

	Jenkins	GHA	CircleCI
Runs where	In-cluster	GitHub-hosted	CircleCI-hosted
Build tool	kaniko	buildx	docker
Secrets	k8s Secret	repo + env secrets	Contexts
Approval	<code>input</code> step	Environment	<code>type: approval</code>
Trigger	“Build Now”	<code>workflow_dispatch</code>	UI / API trigger
Touches cluster	helm install / kubectl	Git commit only	Git commit only

Jenkins is the one that touches the cluster directly (it runs *in* the cluster, after all). The two cloud CIs only commit to Git. Both patterns work; the GitOps-only pattern is cleaner separation of concerns.

17.9 What iteration 7 *doesn't* have

- No multi-region.
- No artifact signing (cosign, Notary).
- No SBOM generation.

These would be the next steps for a real production pipeline. We cover them as gaps in [Chapter 20](#).

Part III — Advanced Topics

Chapter 18: Observability

18.1 Concept — three pillars (and a fourth)

+-----+	+-----+	+-----+
Logs	Metrics	Traces
(events)	(timeseries)	(req path)
+-----+	+-----+	+-----+
	+-----+	
	Profiles	<-- fourth pillar (newer)
	(CPU/mem	
	flame)	
	+-----+	

- **Logs** — discrete events, often unstructured. Postgres logs, app stdout, kernel messages.
- **Metrics** — numbers over time. CPU 23%, requests/sec, error rate.
- **Traces** — request paths across services. “this 500ms came from 400ms in DB, 80ms in backend, 20ms in network.”
- **Profiles** — what code burned the CPU? What allocated the memory?

Most platforms ship the first three. Profiling is the new frontier (Pyroscope, Parca, Grafana Cloud Profiles).

18.2 Why it matters

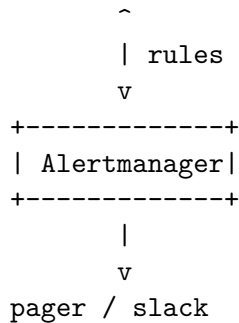
Without observability:

- “It’s slow” → “I don’t know why.”
- “Users are seeing 500s” → “I don’t know which service.”
- “Deploy seems fine” → “...and we don’t notice the regression for two days.”

Observability collapses the time-to-understand and is the difference between a platform people trust and one they don’t.

18.3 The Prometheus model

+-----+	/metrics	+-----+	PromQL	+-----+
target	<-----	Prometheus	<-----	Grafana
(app)	pull	(TSDB)	query	UI
+-----+		+-----+		+-----+



- **Pull-based** — Prometheus scrapes targets' `/metrics` endpoints on a schedule.
- **Service discovery** — in k8s, ServiceMonitor and PodMonitor CRDs tell Prometheus what to scrape.
- **PromQL** — the query language. Powerful but spiky learning curve.
- **Alertmanager** — handles routing, deduplication, silencing.

18.4 Key metric types

Type	Example	When to use
Counter	<code>http_requests_total</code>	Things that only go up (resettable on restart)
Gauge	<code>memory_bytes</code>	Things that go up and down
Histogram	<code>request_duration_seconds_bucket</code>	Distributions (p50/p99)
Summary	quantiles computed in-app	Like histogram but client-side

Rule of thumb: prefer histograms over summaries — they aggregate across instances correctly.

18.5 The RED method and the USE method

Method	For	Track
RED	Services	R ate, E rrors, D uration
USE	Resources	U tilization, S aturation, E rrors

Both fit on one Grafana dashboard. If you remember nothing else: rate, errors, duration for every service; utilization, saturation, errors for every node and disk.

18.6 Istio + Prometheus in our lab

Envoy sidecars emit metrics like:

- `istio_requests_total{response_code, source_workload, destination_workload}` — counter.
- `istio_request_duration_milliseconds_bucket` — histogram.
- `envoy_cluster_upstream_cx_active` — connections.

Our PodMonitor scrapes them. Grafana dashboards (Istio Mesh, Istio Service, Istio Workload — they ship as dashboard IDs 7639/7636/7630) give you all the visualization out of the box.

18.7 Logs — what we don't have (yet)

The lab doesn't ship a logging pipeline. AKS sends container stdout to Log Analytics via the OMS agent, which is enough for “give me the last 100 lines of pod X” but not for fleet-wide queries.

For real systems, the path is:

```
pod stdout -> node agent (Fluent Bit/Vector/Promtail) -> backend
                                                    (Loki / ELK / Splunk / Azure Monitor)
```

We'd add Loki + Promtail for a Grafana-native stack, or Fluent Bit + Elasticsearch for the heavier option.

18.8 Tracing — also not in the lab

Istio can emit Zipkin/Jaeger spans for free. Adding a tracing backend (Jaeger, Tempo) would close this gap. For interview talking points: “we'd add Tempo, configure Istio to sample 10%, and trace request paths through frontend → backend → DB.”

18.9 SLOs and error budgets

A **Service Level Objective** is a target: “99.9% of requests should be successful.” A **Service Level Indicator** is the measurement: actual success rate. An **error budget** is $100\% - \text{SLO}$ — the amount of failure you're allowed.

```
SLO:                99.9%
30-day window:      43,200 minutes
Error budget:       43,200 * 0.001 = 43.2 minutes of "down"
```

When the budget is exhausted, you stop shipping risky changes. When you have budget left, you can move faster. **This converts reliability from a vibes-based to a numbers-based decision.**

18.10 Interview talking points

Q: “How would you measure the health of a service?”

“RED method: request rate, error rate, duration. Plot all three. The error rate alone misses ‘silently slow.’ The duration alone misses ‘failing.’ The rate alone misses ‘nobody’s calling it.’ Three signals together tell the whole story.”

Q: “Pull vs push for metrics?”

“Prometheus is pull. The advantages: targets don't need to know where the metrics backend is, dead targets get noticed (scrape fails), and security — the backend opens connections, not the apps. Disadvantages: doesn't fit short-lived jobs (Pushgateway exists for those), and harder for jobs behind NAT. Most workloads should be pull. Cron jobs and serverless = push.”

Q: “What’s the difference between SLO and SLA?”

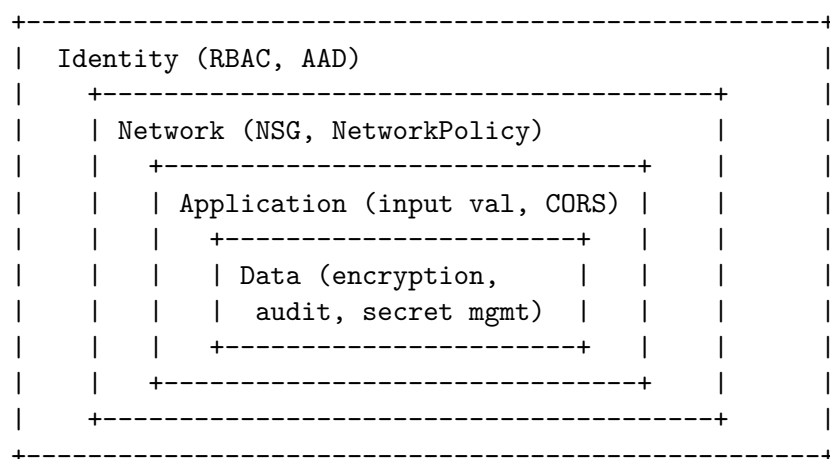
“SLO is internal — what you aim for. SLA is contractual — what you owe customers, usually with refunds attached. SLA is always weaker than SLO so you have margin. Define SLOs to drive engineering decisions; SLA is the lawyer-facing artifact.”

18.11 Exercises

1. Write a counter that increments on every request. Scrape with Prometheus.
 2. Write a histogram for request duration. Query the p99.
 3. Add a PrometheusRule alert: “error rate > 5% for 5 minutes.”
 4. Create a Grafana dashboard with three panels: rate, errors, duration.
 5. Define an SLO (99.5%) and burn-rate alerts (fast and slow burn).
 6. Add Loki, ship logs, query “all logs from one pod in the last hour.”
-

Chapter 19: Security

19.1 Defense in depth



Each layer fails some day. Defense in depth means a breach of one layer doesn't compromise the whole system.

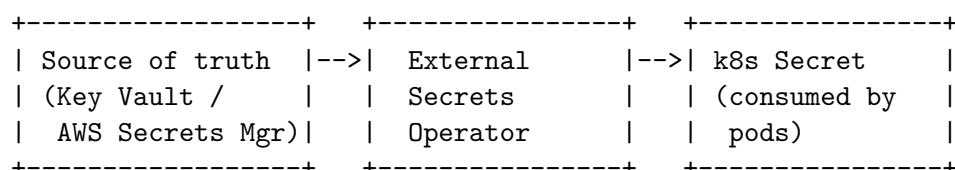
19.2 The least-privilege rule

For every principal — user, SP, MI, pod — the question is: “What’s the smallest set of permissions this needs?” Default to **none**, add **what’s required**, never more.

Practical rules:

- ServiceAccounts for pods, not the **default** SA.
- Roles, not ClusterRoles, unless cluster-scoped.
- Read-only when read suffices.
- Time-bound when possible (PIM in Azure).

19.3 Secrets — the eternally-difficult problem



The pattern: secrets live in a vault. An operator syncs them into k8s as Secret objects. Apps read Secret objects normally. **Secrets never live in Git.**

Kubernetes Secret is base64, not encryption

```
echo -n "supersecret" | base64
# c3VwZXJzZWNyZXQ=      <-- not "encryption", reversible by anyone
```

By default, etcd stores Secrets in plaintext. Enable **etcd encryption at rest** on real clusters. AKS does this for you by default.

What we do in the lab

For lab purposes, Postgres password is in `values-prod.yaml` (and `.gitleaksignore` covers a textbook example). In a real environment we'd:

1. Store the password in Azure Key Vault.
2. Use External Secrets Operator or CSI Secret Driver.
3. Pods mount the Secret as files; never as env vars (env can leak via `/proc`).

19.4 mTLS and zero trust

```
Without mTLS:    pod-A --TCP--> pod-B
With mTLS:       pod-A --TLS(cert==SVID)--> pod-B
                  ^                               ^
                  |                               |
                identity = SA                   verifies cert + identity
```

Istio gives us mTLS for free. In **STRICT** mode, every pod-to-pod call inside the mesh requires a valid client cert. We use **PERMISSIVE** for the lab (allows non-mesh callers) but **STRICT** is the production target.

Zero trust means: don't assume "inside the network" = trustworthy. Authenticate every hop.

19.5 Image security

Practice	What it does
Pin to digest (@sha256:...)	Prevents tag mutation attacks
Sign images (cosign)	Verify what was built is what's running
SBOM (syft, Trivy)	Know what's <i>inside</i> the image
Scan for CVEs (Trivy, Gype)	Catch known issues
Distroless / minimal bases	Smaller attack surface
Pull from private registry	No public-registry rate-limit / takeover

We do: distroless-ish (alpine), Trivy scan (report-only), private ACR. We don't yet: cosign, SBOM publication.

19.6 NetworkPolicy

By default, all pods can reach all pods. NetworkPolicy adds firewalls at the pod level.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-allow-frontend
  namespace: app-prod
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend }
    ports:
    - port: 8080
```

This says: backend pods only accept ingress from frontend pods on 8080. Everything else is rejected.

Gotcha: NetworkPolicy needs a CNI that supports it. kubernetes on AKS has limited support. Azure CNI + Calico / Cilium is the typical setup.

19.7 Pod Security Standards

Three levels:

Level	What's allowed	When to use
privileged	Anything	System pods / Day-0 only
baseline	Most stuff, no obvious unsafe	Default for app namespaces
restricted	Hardened (non-root, RO root FS, no caps)	Production app namespaces

```
apiVersion: v1
kind: Namespace
metadata:
  name: app-prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
```

PSAs replaced the old PodSecurityPolicy (deprecated, removed in 1.25).

19.8 Supply chain — the hot topic

```
code -> commit -> CI build -> image -> registry -> cluster
  ^         ^         ^         ^         ^         ^
  |         |         |         |         |         |
```

signed	PRs	isolated	signed	scanned	admission
commits	reviewed	runners	images	on push	controller verifies
					sigs (Kyverno/Sigstore)

Each arrow is a place to inject something nasty (SolarWinds, codecov, etc). Production-grade supply chain controls all of them. Cosign + Sigstore + Kyverno admission policies is the modern stack.

We don't have this in the lab. Documented as a gap.

19.9 In our lab (security summary)

- **Identity:** SP with bounded scope; AKS uses MI to pull from ACR.
- **Network:** NSGs at subnet level; no NetworkPolicy yet.
- **mTLS:** Istio PERMISSIVE (not STRICT).
- **Secrets:** values files for the lab (gitleaks scans, ignore file for false positives).
- **Image:** Trivy scan (report-only). No signing.
- **Cluster:** AKS managed by Azure (etcd encrypted, control plane patched).

19.10 Interview talking points

Q: “How do you manage secrets in Kubernetes?”

“Not in YAML. Use a vault — Key Vault, AWS Secrets Manager, HashiCorp Vault — as the source of truth. Sync into k8s Secret objects via External Secrets Operator or CSI Secret Driver. Enable etcd encryption at rest. Mount as files, not env vars. Rotate via the vault (re-sync triggers pod restart). The Git repo never sees a real secret.”

Q: “What is mTLS and when would you use it?”

“Mutual TLS — both client and server present certs. Typical TLS only authenticates the server. Use mTLS for service-to-service when you want zero trust: every hop authenticates. Service mesh (Istio, Linkerd) automates the cert issuance so you don't manage them per app. Performance cost is negligible at scale; the main work is making sure your identity model (cert == SA) lines up with your authorization model.”

Q: “What's the principle of least privilege in k8s?”

“Each ServiceAccount has only the permissions it strictly needs. The default SA in a namespace should have nothing. Roles, not ClusterRoles, unless cluster-scope is required. Read-only when reads suffice. For RBAC reviews, periodically audit `kubectl auth can-i --as=sa:ns:name --list` per SA.”

19.11 Exercises

1. Find a Pod running as `default` SA in your cluster. Move it to a dedicated SA with no permissions.
2. Add a NetworkPolicy that defaults-deny in a namespace. Add an explicit allow for what should work.
3. Enable the `restricted` PSA on a namespace. Watch which pods refuse to start.

4. Sign an image with cosign. Verify.
 5. Configure Kyverno to reject unsigned images.
 6. Set up External Secrets to pull a Key Vault secret into a k8s Secret.
-

Chapter 20: Production-Grade Gaps

20.1 What this lab is not

The lab is intentionally small. Things you'd add or change for production:

20.2 High availability

Gap	What we have	What prod needs
Cluster nodes	1 node (B2s)	3+ nodes across AZs
Database	StatefulSet, 1 replica	Managed Postgres with HA, backups
Region	Single (eastus2)	Active/passive or active/active
Load balancer	Single public LB	Front Door / global LB

20.3 Storage

Gap	What we have	What prod needs
Persistent volumes	<code>managed-csi</code> LRS disk	ZRS or premium SSD, snapshotted
Backups	None	Velero or Azure Backup, tested restores
Disaster recovery	None	Geo-replicated, restore runbook

20.4 Security

Gap	What we have	What prod needs
Image signing	None	cosign + admission verification
Secrets in repo	Some (lab)	All secrets in Key Vault
Network isolation	Public LB	Private cluster, private endpoints
Audit log retention	Default Azure	Centralized SIEM (Sentinel)
Pod Security	None set	restricted PSA on app namespaces
NetworkPolicy	None	Default-deny per namespace

20.5 Observability

Gap	What we have	What prod needs
Metrics	Prometheus	Long-term storage (Mimir, Cortex)
Logs	Container stdout	Loki/ELK with retention
Traces	None	Tempo/Jaeger with sampling
Alerts	A few rules	Full SLO + burn-rate alerts
Synthetic monitoring	None	Pingdom/Azure Application Insights
On-call rotation	None	PagerDuty / Opsgenie integration

20.6 Deployment

Gap	What we have	What prod needs
Blue/green	Yes (frontend, backend)	Same + canary for low-risk changes
Analysis	Success-rate + min-traffic	+ latency p99, + business metrics
Rollback	Manual <code>helm rollback</code>	Automated via failed analysis
Multi-env promote	dev -> prod	dev -> staging -> prod with bake time
Feature flags	None	LaunchDarkly / Flagsmith / Unleash

20.7 Compliance

Gap	What we have	What prod needs
Audit logs	Default Azure	Tamper-resistant retention (SOC2/ISO)
Encryption	At rest by default	+ customer-managed keys (CMK)
Access reviews	None	Quarterly attestation via PIM
Change management	Git PRs	+ JIRA tickets + approvals
Vulnerability mgmt	Trivy report	Tracked CVEs with SLA per severity

20.8 Cost

Gap	What we have	What prod needs
Right-sizing	Eyeballed	VPA recommendations + monthly review

Gap	What we have	What prod needs
Idle	Run-and-destroy	Spot nodes for non-critical; cluster autoscaler
Cost attribution	None	Tags + Kubecost / Azure Cost Mgmt

20.9 How to discuss the gaps in an interview

The trap: claiming the lab is production-ready. Don't.

The good answer: **“Here’s what I built, here’s what I’d add for production, and here’s roughly the order I’d add it in based on risk.”**

Sample priorities:

1. Multi-AZ nodes + managed Postgres (HA).
2. Secrets in Key Vault + External Secrets.
3. NetworkPolicy default-deny.
4. Image signing + admission verification.
5. Centralized logging.
6. Backups + restore test.

Each of these is a 1-2 day project in isolation. The lab is the starting position.

Chapter 21: Interview Questions and Answers

This chapter is a curated list of platform engineering questions with concise, structured answers. Use it for warm-up and self-quizzing.

21.1 Cloud and Azure

Q: “Walk me through how a request reaches a pod in your AKS cluster.”

DNS resolves the public hostname to the Azure Load Balancer IP. The LB sends the packet to a node port. kube-proxy (iptables/IPVS) rewrites the destination to a pod IP via the Service’s Endpoints. In our lab Istio’s ingress gateway is the first pod hit; it routes (via VirtualService) to the right backend Service, which Endpoints to the workload pods.

Q: “What’s the difference between a system and user node pool?”

System pool runs cluster-critical pods (CoreDNS, metrics-server, omsagent). It’s required, has taints AKS manages for you, and should not run application workloads. User pools are for your apps — you can add many, with different SKUs, OS, taints, or autoscaling settings.

Q: “Why might you choose Azure CNI over kubenet?”

Pod IPs come from the VNet, so they’re directly addressable from other Azure resources without NAT. Required for advanced features like Azure Private Endpoints to pods, Calico/Cilium network policies, and predictable performance. Cost: heavier IP usage — every pod gets a VNet IP.

21.2 Kubernetes

Q: “What happens when you `kubectl apply` a Deployment?”

kubectl sends the YAML to the API server (after auth/authz/admission). API server writes to etcd. The Deployment controller notices, creates/updates a ReplicaSet. ReplicaSet controller creates pods. Scheduler binds pods to nodes. kubelet on each node sees the binding, pulls images, starts containers, reports back. Status is reconciled back up the chain.

Q: “What’s an Operator? How is it different from a controller?”

Every Operator is a controller. The distinction is intent: an Operator codifies *operational knowledge* about a specific application — installing it, configuring it, backing up, upgrading, failing over.

Postgres operators run Postgres. Prometheus Operator manages Prometheus. A plain controller might just manage a CRD without app-specific knowledge.

Q: “How do you debug ‘service is unreachable from another pod’?”

- (1) `kubectl get svc` — does the Service exist, right port, right selector? (2) `kubectl get endpoints` — are there endpoints? Empty endpoints = selector matched no pods. (3) From a pod: `nslookup <svc>.<ns>.svc.cluster.local`. (4) `curl http://<svc>:<port>`. If DNS fails: CoreDNS issue. If DNS works but TCP fails: NetworkPolicy, kube-proxy, or the service’s containers aren’t listening. Tools: `kubectl debug node/...` and `nsenter` are the next stop.

21.3 GitOps and CI/CD

Q: “Push-based vs pull-based deployment — which would you pick?”

Pull-based (GitOps). Cluster credentials never leave the cluster, CI doesn’t need cluster creds, and drift remediation is built in (selfHeal). The trade-off is feedback latency — CI doesn’t directly know when the deploy succeeded; you observe it via ArgoCD’s status. For most teams the security + drift benefits outweigh the latency. We use pull-based with ArgoCD in this lab.

Q: “What’s a blue/green deployment?”

Run two identical environments — current (blue) and new (green). Switch traffic atomically at the load balancer. If green has issues, switch back. Cost: double the resources for the cutover window. Benefit: zero-downtime, instant rollback. In our lab Argo Rollouts manages this — the `activeService` and `previewService` objects point at different sets of pods, and the active selector flips on promotion.

Q: “How do you decide a deployment is healthy enough to promote?”

A signal that combines (a) the application liveness — pods are up, ready, not crashing — and (b) production traffic behavior — error rate stays within SLO, latency p99 stays within SLO. In our lab we use Argo Rollouts AnalysisTemplate with two PromQL metrics: minimum traffic and success rate. Both must pass.

21.4 Networking

Q: “Explain the difference between a NodePort, LoadBalancer, and Ingress Service.”

NodePort opens a port on every node and forwards to the Service. Cluster-external but you have to know a node IP and the high-numbered port. LoadBalancer in cloud k8s provisions a real LB and gives you a public IP. One per Service is expensive. Ingress is an L7 router — one IP, many hostnames/paths, TLS termination — typically backed by a single LoadBalancer Service for the ingress controller (nginx, Istio gateway).

Q: “What is mTLS and when would you use it?”

(See 19.10.)

Q: “Why might pod-to-pod traffic break in AKS?”

NSG explicit DenyAllInBound at a priority that overrides the implicit AllowVnet rule. Or NetworkPolicy without explicit allow. Or CoreDNS misconfiguration. Or a Service with mismatched

selector / no endpoints. Or the receiving container not actually listening on 0.0.0.0 (only listening on 127.0.0.1).

21.5 Terraform

Q: “How do you handle Terraform state?”

(See 7.9.)

Q: “What’s the difference between `count` and `for_each`?”

`count = N` gives you N identical copies indexed by integer. `for_each = {...}` gives you copies keyed by map key. If you ever remove an element from the middle of a count list, every later element shifts and gets destroyed/recreated. `for_each` is keyed and stable. **Rule of thumb:** count only for “I want N of the same thing,” `for_each` for everything else.

Q: “How do you migrate Terraform state from local to remote?”

- (1) Configure the `backend` block.
- (2) Run `terraform init -migrate-state` — Terraform copies the local state to the backend.
- (3) Verify with `terraform plan` (should be no changes).
- (4) Delete the local `terraform.tfstate` so it can’t be edited accidentally.

21.6 Observability

Q: “RED vs USE method?”

(See 18.10.)

Q: “What’s a histogram in Prometheus and why prefer it over a summary?”

Histogram exposes pre-defined buckets (e.g., “requests in <0.1s”, “<0.25s”, ...). You compute quantiles at query time with `histogram_quantile()`, and you can aggregate across instances. Summary computes quantiles client-side per instance — accurate but not aggregatable. Prefer histogram for distributed services; summary is for cases where per-instance accuracy matters and aggregation doesn’t.

Q: “How do you alert on SLO burn rate?”

Two alerts: a **fast burn** (e.g., “burning 14× over 1h”) and a **slow burn** (e.g., “burning 3× over 6h”). Fast catches major incidents quickly; slow catches sustained degradation. The math: with a 99.9% SLO and a 30-day window, a 14× burn over 1h consumes 2% of the monthly budget — page-worthy.

21.7 Security

Q: “How would you secure a service-to-service call inside a cluster?”

mTLS via service mesh (Istio STRICT mode) plus NetworkPolicy default-deny with explicit allows. Mesh handles cryptographic identity; NetworkPolicy handles L4 access control. Belt and suspenders. Identities map to ServiceAccounts; authorization policies (Istio AuthorizationPolicy) say which identity can call which.

Q: “What’s the blast radius of a leaked SP secret?”

Whatever the SP can do at its scope. If it's Contributor on a subscription, everything in that subscription. **Mitigations:** rotate immediately, use Managed Identity instead, restrict SP to least scope (single RG), require AAD PIM for elevation. Detection: Microsoft Defender for Cloud + Activity Log alerts on unusual SP usage.

Q: “How do you handle a critical CVE in a base image you depend on?”

- (1) Identify all images using it (image inventory + SBOM helps).
- (2) Check if the vuln is exploitable in your context (the CVSS isn't your context).
- (3) If exploitable: emergency rebuild and deploy with patched base.
- (4) If not exploitable: file ticket, fix on regular cadence.
- (5) For unfixable (alpine apk gaps), document the accepted risk and add a workaround (e.g., front the app with a WAF rule).

21.8 Behavioural / system-design adjacent

Q: “Tell me about a time you saw drift in production.”

(Your specific story. Mine: an NSG was edited in the portal during an incident, then Terraform later overwrote it on a routine `apply`. Lesson: terraform plan in CI for every PR, alert on drift via `terraform plan -detailed-exitcode`, and treat the portal as read-only.)

Q: “How would you design a platform for 20 product teams to deploy 50 services?”

Multi-tenant cluster(s) with namespaces per team. ArgoCD with AppProjects per team for RBAC. A platform Helm library — internal chart for the standard service shape (Deployment + Service + VirtualService + PodMonitor). Self-service: a “deployer” repo each team owns, pointed at by ArgoCD. Guardrails: PSA **restricted**, NetworkPolicy default-deny, image admission policy. Observability: shared Prometheus + Grafana, dashboards generated from the chart. Cost attribution by namespace.

Q: “What would you do in the first 30 days of a platform engineering role?”

- (1) Read existing IaC and inventory infra.
 - (2) Pair with on-call to learn pain points.
 - (3) Inventory deploys: who deploys what, how, how often.
 - (4) Define platform SLO.
 - (5) Pick *one* small win that reduces friction (faster CI, drift detection, secret rotation) and ship it.
 - (6) Document something undocumented. Build trust before redesigns.
-

Appendix A: Cheatsheets

A.1 kubectl

```
# context / namespace
kubectl config get-contexts
kubectl config use-context <ctx>
kubectl config set-context --current --namespace=<ns>

# get
kubectl get pods                # current ns
kubectl get pods -A             # all namespaces
kubectl get pods -o wide        # with node + IP
kubectl get pod <p> -o yaml      # full spec
kubectl get pod <p> -o jsonpath='{.spec.nodeName}'

# debug
kubectl describe pod <p>
kubectl logs <p> -c <container> # multi-container
kubectl logs <p> --previous      # last restart
kubectl exec -it <p> -- sh
kubectl port-forward svc/<svc> 8080:80

# events
kubectl get events --sort-by='.lastTimestamp'

# rollouts
kubectl rollout status deploy/<d>
kubectl rollout history deploy/<d>
kubectl rollout undo deploy/<d>

# argo rollouts
kubectl argo rollouts list rollouts
kubectl argo rollouts get rollout <r>
kubectl argo rollouts promote <r>
kubectl argo rollouts abort <r>
```

A.2 helm

```
helm install <rel> <chart>
helm install <rel> <chart> -f values.yaml --set foo=bar
helm upgrade --install <rel> <chart> -f values.yaml
helm rollback <rel> <revision>
helm list -A
helm status <rel>
helm get values <rel>
helm get manifest <rel>
helm template <chart> -f values.yaml          # render without applying
helm uninstall <rel>
```

A.3 terraform

```
terraform init                                # download providers
terraform fmt -recursive                      # format files
terraform validate                            # syntax/refs OK?
terraform plan -out=tf.plan                  # preview, save plan
terraform apply tf.plan                      # apply saved plan
terraform apply -auto-approve                # no prompt
terraform destroy
terraform state list
terraform state show <addr>
terraform import <addr> <id>                 # bring existing into state
terraform output                             # show outputs
```

A.4 az

```
az login
az account list -o table
az account set -s <sub-id>

az group create -n <rg> -l <region>
az aks get-credentials -g <rg> -n <cluster>
az acr login -n <reg>
az acr build -r <reg> -t <reg>/<repo>:<tag> .

az ad sp create-for-rbac --name <name> --role Contributor \
  --scopes /subscriptions/<id>
```

A.5 docker

```
docker build -t <name>:<tag> .
docker run --rm -it <name>:<tag>
docker exec -it <container> sh
docker logs -f <container>
```

```
docker images
docker ps -a
docker system prune -af                # nuke unused (careful)
```

A.6 PromQL one-liners

```
# request rate per service
sum by (destination_workload) (rate(istio_requests_total[5m]))

# error rate
sum(rate(istio_requests_total{response_code=~"5.."}[5m]))
  / sum(rate(istio_requests_total[5m]))

# p99 latency
histogram_quantile(0.99,
  sum by (le) (rate(istio_request_duration_milliseconds_bucket[5m])))

# CPU usage by pod
sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))

# memory by pod
sum by (pod) (container_memory_working_set_bytes)
```

Appendix B: Building PDF and DOCX

```
cd docs/textbook
make                # everything
make handbook      # just the handbook
make glossary       # just the glossary
make clean          # remove generated artifacts
```

The Makefile uses the `pandoc/latex:latest` Docker image — no local pandoc or LaTeX install needed. PDF generation uses `lualatex` (handles Unicode better than `pdflatex` but still won't have every emoji glyph; DOCX handles those via Word's font fallback).

If a chapter changes, just re-run `make`. Pandoc rebuilds incrementally.

Appendix C: Where to Go Next

If you’ve worked through the lab and the chapters, here are the natural next steps, in roughly increasing difficulty.

Operational next steps - Add a real logging pipeline (Loki + Promtail). - Add tracing (Tempo) and wire Istio’s Zipkin export. - Replace the Postgres StatefulSet with Azure Database for PostgreSQL Flexible Server. - Move secrets to Azure Key Vault + External Secrets Operator. - Add a NetworkPolicy default-deny on app namespaces.

Platform-engineering next steps - Build a self-service Helm library (a single internal chart for “what a service looks like here”) and migrate the three-tier app to use it. - Add Backstage for service catalog + scaffolding. - Wire up cost attribution via Kubecost or Azure Cost Management. - Multi-cluster with Cluster API (CAPI) or AKS fleet. - Argo Workflows for batch / data jobs.

Career next steps - The **CKA** (Certified Kubernetes Administrator) certification is a 2-hour hands-on exam — directly useful for platform roles. - The **HashiCorp Terraform Associate** is a multiple-choice exam — easy to add once you’ve used Terraform daily. - The **Azure AZ-104** (admin) and **AZ-400** (DevOps) round out the Azure side. - Read *Site Reliability Engineering* (Google), *Database Internals* (Petrov), *Designing Data-Intensive Applications* (Kleppmann). They each rewire how you think about a layer of the stack.

Community - Follow the CNCF landscape (landscape.cncf.io). - Watch KubeCon talks on YouTube — start with the “production stories” track. - Pick one open-source platform project (ArgoCD, Crossplane, Backstage) and read enough of its source to add a tiny feature or doc fix.

Good luck. You’ve already done the hard part — building something real.